Overcoming the Cold-Start Calibration Barrier in Distributed Automotive Radar Re-Simulation via Dual-Horizon Dynamic Alignment and Preflight Seeding

A-PRISM research manuscript
Author and affiliation details withheld pending confirmation
October 2026; technical draft

***Abstract*—Distributed replay must initialize calibration state as well as kinematic tracks. We analyze a dual-horizon alignment schedule, physically qualified observations and provenance-aware preflight seeding, with explicit separation between a proposed estimator and the archived wrapper integration. Exact scalar error products show why a reciprocal gain is an e-folding scale rather than a threshold-convergence guarantee. We derive timestamp deskew, clock-bias identifiability and track-conditioned beat estimation, and document source-level limits. Paired replay counts are reproduced while unverified accuracy and cluster-speedup claims remain clearly identified as reports.**

I Introduction: why replay state is not just tracker state

A distributed replay job has two distinct continuity problems. The object tracker needs sufficient recent measurements to initialize its kinematic state. The radar front end may also carry mounting alignment, clutter statistics, temperature history and qualifier counters. A single warm-up policy is valid only if it covers the slowest state on the path to the scored output. Splitting a 300-log drive across twenty workers gives roughly fifteen scored logs per worker; a preceding log can be replayed without scoring, but its adequacy must be measured in accepted calibration updates rather than file count. The project narrative reports a 16-log chunking attempt and a calibration bottleneck. Its conversion between 400 frames, 30–40 logs and twenty minutes is inconsistent without an independent log-duration definition. At 20 Hz, 400 frames are twenty seconds and 46 frames are 2.3 seconds. We therefore treat the 360,000 dropped-frame and cluster-speedup statements as project reports requiring scheduler and KPI logs, not as measured outcomes established here. The useful research question survives this correction: how should a replay distinguish estimator warm-up from validated state initialization?

I-A System architecture and scope

The Gen8 iND13400 sensor repository contains the BBE32-side signal-processing wrappers and pure C kernels. Its relevant chain is ADC, range FFT, Doppler FFT, CFAR, CDC packing, beamforming, alignment and radar output. The Domain Controller repository contains DC_SIL wrappers and the unified F360 tracker. Preflight header handling and temporal deskew connect these domains. The separate 4D RadarSplat/V&V repository is deferred; no validation result from it is claimed.

II Related work and contribution boundary

Recursive estimation follows Kalman's state-space framework [1]; practical detection-to-track association and motion-model management are established subjects. The proposed schedule is an engineering gain policy, not a new proof of Kalman optimality. The distinct project contribution is a traceable combination of startup scheduling, physically qualified updates, replay-state seeding and guarded integration. The residual-frequency estimator is related to the long-established phase-difference frequency-estimation literature [4]. Neither an exponential schedule nor a stored calibration field alone establishes patent novelty.

III Dual-horizon dynamic alignment

Consider a scalar local alignment observation $z_N = \theta^\star + \eta_N$ and an admitted update with gain K_N:

$$\hat{\theta}_{N+1} = \hat{\theta}_N + K_N(z_N - \hat{\theta}_N). \tag{1}$$

Here $\hat{\theta}_N$ is the estimate, $\theta^\star$ the locally constant mounting angle, and η_N measurement noise. This reduced model is an analysis model; it is not a recovered implementation of the buried production DABB recursion.

$$K_N = K_f e^{-\alpha N} + K_s. \tag{2}$$

The fast component $K_f = 0.05$ and slow component $K_s = 0.0025$ are dimensionless. The archived narrative uses $\alpha = 0.08$ per update; the wrapper and host A/B code use a default of 0.1. These are distinct configurations, not interchangeable measurements.

An update index must advance only under the documented eligibility policy; advancing it on rejected scans changes behavior.

$$e_N = e_0 \prod_{i=0}^{N-1} (1 - K_i), \quad e_N = \hat{\theta}_N - \theta^\star. \tag{3}$$

In the noiseless model and for $0 < K_i < 1$, error contracts monotonically. Because $\log(1-u) \le -u$ for $0 < u < 1$, a convenient upper bound is

$$\frac{|e_N|}{|e_0|} \le \exp\left[-NK_s - K_f \frac{1 - e^{-\alpha N}}{1 - e^{-\alpha}}\right]. \tag{4}$$

All terms are dimensionless; this is an upper bound, not an equality or a guaranteed 46-frame lock. A fixed gain 0.0025 has e-folding time $-1/\log(0.9975) \simeq 399.5$ updates. A requested threshold depends on the initial error and noise, not simply on the reciprocal gain.

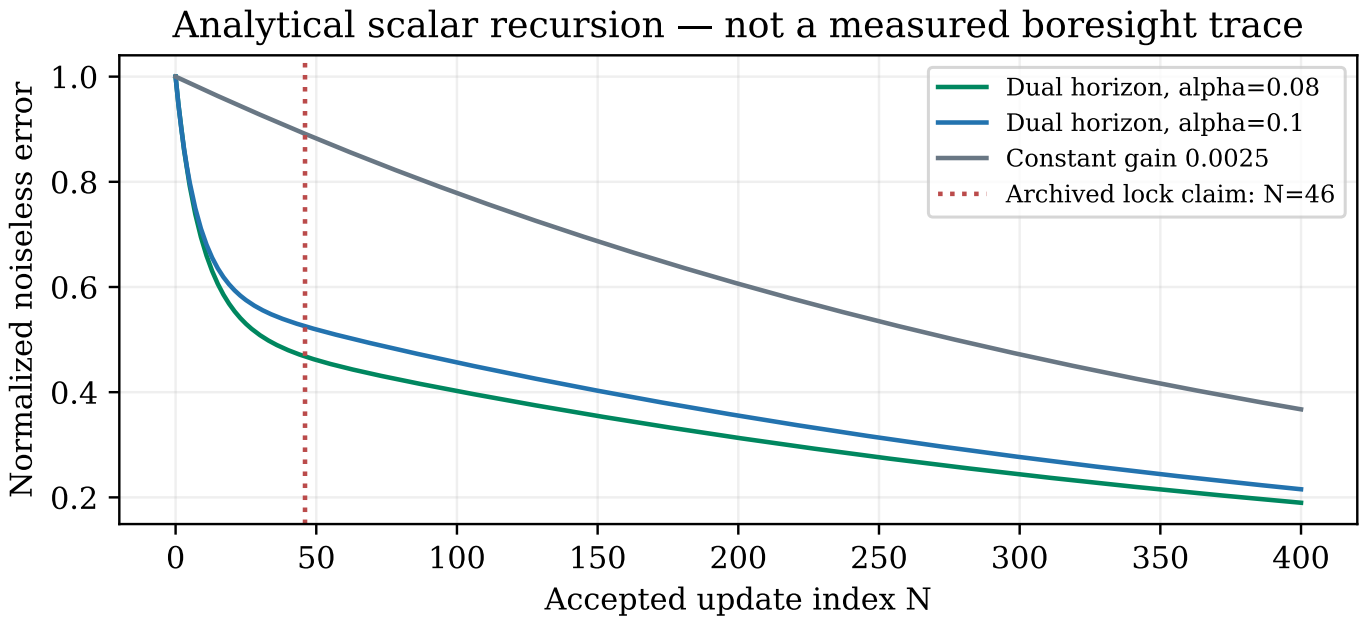


Figure 1: Exact products for the explicitly stated scalar model. The archived frame-46 claim is marked for context, not fitted into the curves.

III-A Noise and lock certificates

For independent zero-mean noise with variance σ_η^2, the scalar covariance obeys

$$P_{N+1} = (1 - K_N)^2 P_N + K_N^2 \sigma_\eta^2. \tag{5}$$

The steady fixed-gain variance is $K_s \sigma_\eta^2 / (2 - K_s)$. A fast initial gain trades noise sensitivity for bias contraction. Ten small successive estimate changes are a stability indicator, not a certificate of absolute alignment: a frozen estimator is stable even if biased. A defensible release test combines observation support, residual statistics, independent reference error where available, and a bounded hold count.

III-B Stationary-clutter qualification

For a stationary world scatterer, a nonrotating sensor moving at ego velocity $\mathbf{v}_e$ predicts radial rate $-\mathbf{u}^T \mathbf{v}_e$. An admissibility gate is

$$|v_r + \mathbf{u}^T \mathbf{v}_e| \le \varepsilon, \quad \mathbf{u} = (\cos\theta, \sin\theta)^T. \tag{6}$$

The radial speed v_r and tolerance ε are in m/s. Sensor lever-arm rotation, sign conventions and aliasing must be handled before this comparison. A reflector can be stationary and therefore legitimately satisfy this gate; the gate does not reject guardrails merely because they are guardrails. No numerical tolerance is invented where the source does not specify one.

IV Preflight seeding and provenance

The header helper accepts azimuth and elevation in degrees within 5 and 2 degrees respectively, with qualifier checks in the transport helper. This validates magnitude and presence, not freshness, sensor identity, finite values or convergence history. Production preflight should bind the seed to sensor serial number, mounting configuration, firmware version, timestamp and quality state. A rejected or stale seed must leave the fallback estimator operational. NaN values require explicit finite checks because ordinary comparisons do not reject them. Historical header injection can avoid estimator initialization delay when the seed is valid. It does not eliminate I/O or replay startup latency, and using a future converged value in a scored online experiment creates look-ahead leakage. Such injection is appropriate for a declared offline replay mode; online cold-start accuracy must be tested without future information.

V Temporal deskew and clock separation

The archived wrapper implements the following explicit coordinate convention:

$$\mathbf{p}_c = R(\omega\Delta t)\mathbf{p} - \mathbf{v}\Delta t - \tfrac{1}{2}\mathbf{a}\Delta t^2. \tag{7}$$

Here position is in metres, velocity in m/s, acceleration in m/s^2, yaw rate in rad/s and signed Δt in seconds. This expression is a specified approximation, not a universal sign convention for every SE(2) time transformation. The rotation maps vectors between the chosen timestamped ego frames; translation must be expressed in a compatible frame. A physical test should use a stationary landmark during known translation and rotation to establish the signs.

$$R(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix}. \tag{8}$$

Its first-order timestamp sensitivity is $\omega R'(\omega\Delta t)\mathbf{p} - \mathbf{v} - \mathbf{a}\Delta t$, so long-range points are particularly sensitive to angular-rate uncertainty.

V-A Yaw-free clock model

Under constant target velocity, let the displaced observation vector be $\mathbf{d} = \mathbf{p} - \tau\mathbf{v}$ while contemporaneous range and range rate satisfy $rw = \mathbf{p}^T\mathbf{v}$. Then

$$rw - \mathbf{d}^T\mathbf{v} = \|\mathbf{v}\|^2 \tau. \tag{9}$$

The range r is in m, radial speed w in m/s, and both sides have units m^2/s. Shared yaw rotations cancel in the inner product. A radial-rate bias b gives $y_k = a_k\tau + r_k b$, with $a_k = \|\mathbf{v}_k\|^2$. Two nondegenerate targets give

$$\begin{bmatrix} \tau \\ b \end{bmatrix} = \begin{bmatrix} a_1 & r_1 \\ a_2 & r_2 \end{bmatrix}^{-1} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}. \tag{10}$$

Identifiability requires $a_1 r_2 - a_2 r_1 \neq 0$ and adequate conditioning. The archived DC update uses an ego-motion proxy, not paired target observations with these physical units; its outputs are a sidecar demonstration rather than a validated clock estimate.

V-B Relative yaw from common displacements

If two sensors measure the same nonzero world displacement $\Delta\mathbf{p}_k$, their local complex displacements obey $D_{ik} = e^{-j\beta_i}\Delta p_k$. Consequently $\arg\sum_k D_{ik}\overline{D_{jk}} = \beta_j - \beta_i$ in the ideal common-pair case. This estimates relative yaw only. Absolute yaw additionally requires surveyed baseline closure or another external reference; zero baseline, negligible displacement and mismatched pairs are failure cases, not calibration successes.

VI Track-conditioned residual beat estimation

For complex fast-time samples x_n at rate f_s, derotate a predicted beat frequency f_p and form

$$z_n = x_n e^{-j2\pi f_p n/f_s}, \quad C_1 = \sum_{n=1}^{M-1} z_n\overline{z_{n-1}}. \tag{11}$$

$$\hat{f}_{\text{res}} = \frac{f_s}{2\pi}\arg(C_1), \quad \hat{r} = \frac{c(f_p + \hat{f}_{\text{res}})}{2\mu}. \tag{12}$$

Here $f_s, f_p, \hat{f}_{\text{res}}$ are in Hz and FMCW chirp slope μ is in Hz/s. This estimates beat frequency and therefore range under the stated model; it is not automatically a slow-time Doppler estimator. The C helper returns cycles/sample and assumes samples were already

derotated: its `f_pred` argument is explicitly unused. The caller owns the phase convention and physical scaling. The separate four-sample cubic stencil computes three adjacent phase products. It is not the same algorithm as arbitrary-length lag-one accumulation. Reported 4.5-fold multiply savings and 0.108 m INT8 RMSE require a specified reference estimator, window length, quantizer, chirp slope and noise distribution; those prerequisites cannot be recovered from a headline alone. Quantization angle bounds require a valid amplitude-to-error ratio, and an arcsine input outside $[-1, 1]$ must be rejected or bounded.

VII Firmware realization and integration status

The exported Gen8 wrapper computes a gain readout and caps update-count configuration at fifty; its own comment says the buried DABB recursion cannot be rewritten by that wrapper. A proposed gain policy is therefore distinguished from a demonstrated production update law. The C kernels are bounded and heap-free at their local interfaces; that observation says nothing about whole-system stack usage or timing. Build definitions connect the new libraries, while compile-time guards select behavioral paths. Guarding code is a rollback mechanism, not evidence of zero deployment risk.

VIII Experimental evidence and its limits

The strongest directly inspectable artifacts are the paired CSV outputs of an instrumented replay harness. They contain scan identifiers, track and detection counts, association counts, assimilation marks and a sum of longitudinal track velocities. They do not contain independently surveyed target trajectories, classification labels, position errors or recall denominators. Accordingly, changes in these counters are observations about the harness, not estimates of fleet accuracy or false-alarm probability. The harness additionally constructs pairing candidates using azimuth and range proximity; such labels are hypotheses rather than truth annotations. The association-count difference is -347, or -3.66%. It is not a verified reduction in false associations, because the CSV

Table 1: Paired replay ledger, independently summed from CSV files

Quantity	OFF	ON
Replay records	155	155
Valid detections	71,779	71,779
Association events	9,478	9,131
Assimilation marks	0	10,130
Records with marks	0	151

files do not distinguish correct and incorrect matches. Marks may be counted at an instrumented update site and should not be equated automatically to distinct reflected targets. Repeated targets and repeated use of a detection must be audited separately. The recorded mean and maximum longitudinal velocity shifts, 0.910 and 3.743 m/s, are changes relative to OFF, not velocity errors relative to truth. The archive reports parent position RMSE 1.969 to 0.651 m, spray RMSE 0.911 to 0.315 m, fixed/adaptive cut-in recall 86.3/99.6%, and IMM recall 88.0%. Those figures are retained as reported benchmark claims. Their truth trajectories, sample definitions and executable scoring pipelines are not provided by the paired CSVs. They are therefore not demonstrated fleet accuracy results in this manuscript. The reported 67.0% parent-RMSE reduction is the arithmetic $1 - 0.651/1.969$; correct arithmetic does not validate the underlying experiment.

VIII-A Host checks and module-level evidence

The archival ledger records 40/40 host checks for both configurations. Equal pass counts establish that the named checks passed; they do not establish binary equivalence, absence of undefined behavior, timing safety, or functional safety. A strict baseline regression comparison requires matching inputs, reset states, compiler options, output fields and tolerances. The four module probes report identity fill (T1), entropy inflation (T2), assimilation marking (T3), and penalty exclusion (T4). In T2, $0.5\exp(2) = 3.694528$ is a deterministic arithmetic probe, not a wet-road accuracy test.

VIII-B Validation design required for a publication claim

Partition complete drives before parameter tuning. Keep sensor poses, calibration history and overlapping temporal windows within a single split to avoid leakage. Establish reflector and target ground truth independently of the algorithm under evaluation. Report position and velocity errors in common coordinates, per-target association precision and recall, and duplicate-update rates. Aggregate confidence intervals at drive or target-trajectory level rather than pretending adjacent scans are independent. Publish ablations for gating, reflector covariance, confirmation, seeding and geometry separately. A hardware deployment claim additionally needs the production DSP build and measured worst-case execution time.

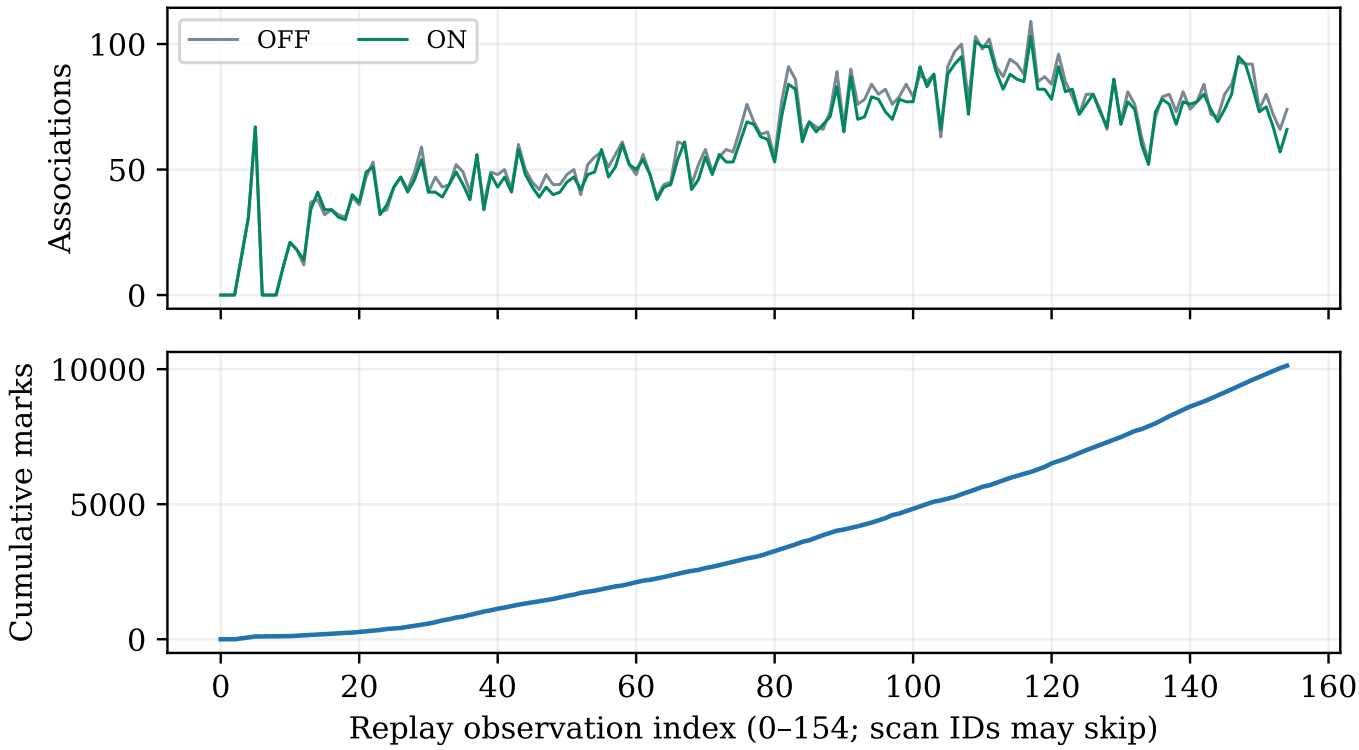


Figure 2: Actual paired replay association series and cumulative ON assimilation marks. No random series or interpolated per-scan observations are used.

IX Conclusion

The project exposes useful estimator and interface design problems, but reproducibility requires a strict separation between equations, implemented call sites and measured evidence. The source supports inspection of guarded kernels and replay counters; it does not by itself support universal lock-time, safety or fleet-accuracy guarantees. The next decisive experiment is an independent ground-truth evaluation with complete configuration and per-drive ablations.

References

[1] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," Journal of Basic Engineering, vol. 82, pp. 35–45, 1960, doi:10.1115/1.3662552.

[2] H. A. P. Blom and Y. Bar-Shalom, "The interacting multiple model algorithm for systems with Markovian switching coefficients," IEEE Transactions on Automatic Control, vol. 33, pp. 780–783, 1988, doi:10.1109/9.1299.

[3] H. Rohling, "Radar CFAR thresholding in clutter and multiple target situations," IEEE Transactions on Aerospace and Electronic Systems, vol. AES-19, pp. 608–621, 1983, doi:10.1109/TAES.1983.309350.

[4] S. M. Kay, "A fast and accurate single frequency estimator," IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 37, pp. 1987–1990, 1989, doi:10.1109/29.45547.

[5] A-PRISM project, "Master archival backup," one.md, October 2026. Internal source/equation/claim ledger; not an independently reviewed publication.

[6] A-PRISM project, paired instrumented replay CSVs, real_off.csv and real_on.csv. Source files and hashes reproduced in the extended reproducibility appendix.

APPENDIX B: SOURCE-TRACEABLE REPRODUCIBILITY PACKAGE

This appendix is a source dossier rather than additional experimental results. It preserves exact archived code and complete unified patches so a reviewer can inspect behavior rather than rely on prose. The complete two-repository patches are retained because each paper depends on the common sensor/controller boundary. A code block appearing in both a full-file archive and a patch serves two different reconstruction tasks: reviewing a standalone kernel and applying an integration delta. Neither is evidence of a fresh successful build. Line breaks in listings may wrap for page width; source bytes are described by hashes.

Artifact manifest and provenance

one.md: 146148 bytes. SHA-256:
`9a2e1e7fcc2b168ede019eced212bd7c`
`2e672fd6d6210f8bb0c174cc85f2db90`
APRISM_Gen8_Full.patch: 33663 bytes. SHA-256:
`94f8df0984220c64fb6831cbd5da1025`
`5c455629c89ef799fb9e0c49ae07e7be`
APRISM_DC_OT_Full.patch: 52306 bytes. SHA-256:
`1fb43249b8ab80b49319bdf4ff2cfc28`
`f3be31eb43f7a8eb2c2344036e9cf0b6`

Fresh-machine reconstruction protocol

Obtain the original private remotes and exact baseline commit identifiers from the project owner; the archive contains placeholders rather than real clone URLs and immutable base SHAs. A branch name alone is not a reproducible base. Clone authorized baselines, check out those identifiers, inspect patch paths, run git apply –check, and only then apply each corresponding patch. New files already tracked in a baseline will not necessarily be carried by an ordinary working-tree diff. Verify the standalone listings against restored source before rebuilding. Record compiler versions and flags; do not infer production DSP qualification from a host C++ build. Replaying the original data additionally requires the original recordings, configuration and parser definitions. The source archive is not a substitute for those input data.

Paired replay table

The following is the complete actual counter series, indexed by the recorded scan identifier. It is not a ground-truth evaluation. OFF and ON input counts are checked for matching scan IDs. Columns give live tracks, detections, association counts and ON marks.

Scan	Tracks	Detections	OFF	ON	Marks
24565	0	534	0	0	0
24566	0	500	0	0	0
24567	0	509	0	0	0
24568	60	496	16	15	30
24569	79	508	31	31	36
24570	99	510	67	67	35
24706	7	532	0	0	1
24707	7	395	0	0	0
24708	7	385	0	0	1
24709	16	368	11	11	6
24710	24	395	21	21	7
24711	35	385	18	18	13
24712	41	400	12	14	11
24713	54	362	37	34	11
24714	63	408	38	41	20
24715	70	394	32	34	13
24716	79	404	34	34	19
24717	91	386	32	31	19
24718	98	384	31	30	14
24719	109	392	39	40	10
24720	118	434	36	37	23
24721	126	396	47	49	20
24722	133	409	53	51	34

Scan	Tracks	Detections	OFF	ON	Marks
24723	146	385	33	32	20
24724	153	424	34	36	37
24725	160	378	43	43	16
24726	165	421	47	47	23
24727	169	394	42	41	39
24728	177	411	49	46	46
24729	181	401	59	54	25
24730	186	396	41	41	51
24731	190	370	47	41	52
24732	184	412	43	39	66
24733	183	375	44	44	43
24734	186	400	52	49	63
24735	188	404	49	44	36
24736	191	406	41	38	62
24737	191	364	55	56	63
24738	191	387	36	34	60
24739	191	393	49	48	46
24740	194	408	48	43	59
24741	194	404	50	47	43
24742	193	372	42	41	57
24743	191	412	60	58	51
24744	194	399	50	48	42
24745	193	356	45	43	39
24746	195	392	42	39	37
24747	196	422	48	43	50
24748	199	402	44	40	41
24749	197	400	44	41	61
24750	197	405	48	45	54
24751	199	412	50	47	46
24752	197	384	40	42	68
24753	200	404	52	48	38
24754	200	437	55	49	41
24755	203	457	57	58	53
24756	206	411	51	47	59
24757	213	409	56	51	46
24758	216	423	61	60	34
24759	224	461	52	52	64
24760	224	414	48	50	57
24761	226	437	56	54	58
24762	222	434	48	48	32
24763	229	439	39	38	48
24764	229	441	44	43	54
24765	233	421	45	44	63
24766	232	467	61	54	62
24767	239	456	60	61	53
24768	239	450	44	42	50
24769	236	422	52	46	36
24770	235	482	58	55	70
24771	232	426	49	48	45
24772	234	457	55	56	65
24773	237	421	58	53	71
24774	242	454	57	53	47
24775	244	456	66	61	63
24776	247	498	76	69	65
24777	246	427	69	68	45
24778	247	442	64	63	60
24779	247	432	65	62	81
24780	250	433	55	53	82
24781	248	466	77	71	73
24782	250	471	91	84	91

Scan	Tracks	Detections	OFF	ON	Marks
24783	250	493	86	82	86
24784	250	465	64	61	96
24785	244	445	69	69	59
24786	250	479	67	65	100
24787	250	450	66	68	89
24788	250	471	73	71	89
24789	249	462	89	83	76
24790	250	438	67	65	42
24791	250	457	90	87	66
24792	250	469	76	70	54
24793	250	465	78	71	63
24794	250	476	84	79	74
24795	250	500	80	78	79
24796	250	501	82	73	86
24797	250	454	76	70	113
24798	250	471	79	78	58
24799	250	495	84	77	87
24800	250	485	79	77	84
24801	250	456	88	91	84
24802	250	471	85	83	91
24803	250	526	88	88	94
24804	250	469	63	66	46
24805	250	492	91	88	69
24806	250	511	97	92	62
24807	250	492	100	95	98
24808	250	463	77	72	83
24809	250	554	103	101	91
24810	250	540	98	99	101
24811	250	526	102	99	54
24812	250	539	91	89	94
24813	250	486	87	82	84
24814	250	522	94	88	96
24815	250	544	92	86	82
24816	250	525	88	85	69
24817	250	533	109	103	70
24818	250	521	85	82	92
24819	250	520	87	82	95
24820	250	570	84	78	131
24821	250	529	96	91	82
24822	250	526	85	81	89
24823	250	523	79	82	107
24824	250	522	72	72	114
24825	250	514	80	76	96
24826	250	494	80	80	95
24827	250	540	74	73	84
24828	250	509	66	67	101
24829	250	499	86	86	94
24830	250	493	69	68	112
24831	250	534	81	77	110
24832	250	537	76	74	113
24833	250	447	63	60	72
24834	250	494	53	52	102
24835	250	548	71	73	109
24836	250	563	79	78	139
24837	250	514	80	76	125
24838	250	533	73	68	118
24839	250	561	81	77	128
24840	250	507	74	76	113
24841	250	519	77	77	91
24842	250	550	84	80	97

Scan	Tracks	Detections	OFF	ON	Marks
24843	250	532	72	74	108
24844	250	568	71	69	114
24845	250	540	80	74	113
24846	250	520	84	80	119
24847	250	557	93	95	130
24848	250	573	92	92	115
24849	250	553	92	83	101
24850	250	524	74	73	110
24851	250	563	80	75	112
24852	250	546	72	67	103
24853	250	546	66	57	110
24854	250	542	74	66	92

Appendix C: Verbatim kernel and interface sources

software/bbe32/dyn_alignment/ghost_calib.c

```
/* A-PRISM ghost calibration implementation. See header for contract. */
#include "ghost_calib.h"

#include <math.h>

static float gc_norm2(const float *v)
{
    return sqrtf(v[0] * v[0] + v[1] * v[1]);
}

GhostYawResult_T GhostEstimateYaw(const float *zd1, const float *zg1,
                                  const float *zd2, const float *zg2,
                                  int32_t m, const float *s1, const float *s2)
{
    GhostYawResult_T out = {0};
    float h_re = 0.0f, h_im = 0.0f;
    float c1[2] = {0.0f, 0.0f}, c2[2] = {0.0f, 0.0f};
    float bx, by, B[2], blen, mean_b[2];
    float ca, sa, rb_re, rb_im;
    int32_t k;
    out.valid = false;
    if ((m <= 0) || (m > GHOST_CALIB_MAXREC)) {
        return out;
    }
    for (k = 0; k < m; k++) {
        float d1x = zd1[2 * k] - zg1[2 * k];
        float d1y = zd1[2 * k + 1] - zg1[2 * k + 1];
        float d2x = zd2[2 * k] - zg2[2 * k];
        float d2y = zd2[2 * k + 1] - zg2[2 * k + 1];
        float L = sqrtf(d1x * d1x + d1y * d1y);
        if (L < 1e-12f) {
            return out; /* L=0: target on reflector */
        }
        if (k == 0) {
            out.lmin = L;
        } else if (L < out.lmin) {
            out.lmin = L;
        }
        /* conj(D_j) * D_i accumulation: H = sum D_i * conj(D_j) */
        h_re += d1x * d2x + d1y * d2y;
        h_im += d1y * d2x - d1x * d2y;
        c1[0] += 0.5f * (zd1[2 * k] + zg1[2 * k]);
        c1[1] += 0.5f * (zd1[2 * k + 1] + zg1[2 * k + 1]);
        c2[0] += 0.5f * (zd2[2 * k] + zg2[2 * k]);
        c2[1] += 0.5f * (zd2[2 * k + 1] + zg2[2 * k + 1]);
    }
    if ((h_re == 0.0f) && (h_im == 0.0f)) {
        return out;
    }
    out.delta = atan2f(h_im, h_re);
    c1[0] /= (float)m;
    c1[1] /= (float)m;
    c2[0] /= (float)m;
    c2[1] /= (float)m;
    /* max |C_j| over sensor-2 records (certificate input) */
    out.cmax = 0.0f;
    {
        int32_t k2;
        for (k2 = 0; k2 < m; k2++) {
            float mx = 0.5f * (zd2[2 * k2] + zg2[2 * k2]);
            float my = 0.5f * (zd2[2 * k2 + 1] + zg2[2 * k2 + 1]);
            float cc = sqrtf(mx * mx + my * my);
            if (cc > out.cmax) {
                out.cmax = cc;
            }
        }
    }
```

```
    ca = cosf(out.delta);
    sa = sinf(out.delta);
    /* R(delta) * mean(C_j): rotation of averaged midpoint */
    rb_re = ca * c2[0] - sa * c2[1];
    rb_im = sa * c2[0] + ca * c2[1];
    mean_b[0] = c1[0] - rb_re;
    mean_b[1] = c1[1] - rb_im;
    B[0] = s2[0] - s1[0];
    B[1] = s2[1] - s1[1];
    blen = gc_norm2(B);
    if (blen == 0.0f) {
        return out; /* zero baseline: relative-only, refuse absolute */
    }
    bx = mean_b[0];
    by = mean_b[1];
    /* beta_i = arg(B * conj(mean_b)): Re = dot, Im = cross(By*bx - Bx*by) */
    out.beta_i = atan2f(B[1] * bx - B[0] * by, B[0] * bx + B[1] * by);
    out.beta_j = out.beta_i + out.delta;
    out.valid = true;
    return out;
}

bool GhostCertify(float eps, float lmin, float cmax, float baseline,
                  float *e_rel_deg, float *e_i_deg, float *e_j_deg)
{
    float rho, e_d, eb, e_i;
    const float rad2deg = 57.29577951308232f;
    if ((lmin <= 0.0f) || (baseline <= 0.0f)) {
        return false;
    }
    rho = 4.0f * eps / lmin + (2.0f * eps / lmin) * (2.0f * eps / lmin);
    if (rho >= 1.0f) {
        return false;
    }
    e_d = asinf(rho < 1.0f ? rho : 1.0f);
    eb = 2.0f * eps + 2.0f * cmax * sinf(e_d / 2.0f);
    if (eb >= baseline) {
        return false;
    }
    {
        float r = eb / baseline;
        e_i = asinf(r < 1.0f ? r : 1.0f);
    }
    if (e_rel_deg != NULL) {
        *e_rel_deg = e_d * rad2deg;
    }
    if (e_i_deg != NULL) {
        *e_i_deg = e_i * rad2deg;
    }
    if (e_j_deg != NULL) {
        *e_j_deg = (e_i + e_d) * rad2deg;
    }
    return true;
}

GhostContact_T GhostInverseContact(const float *s, const float *p,
                                   const float *u, float ell)
{
    GhostContact_T out = {{0, 0}, {0, 0}, {0, 0}, 0, 0, false};
    float ulen, ux, uy, dx, dy, d2, ud, B, t, rho, ax, ay, zx, zy, nz;
    ulen = gc_norm2(u);
    if (ulen == 0.0f) {
        return out;
    }
    ux = u[0] / ulen;
    uy = u[1] / ulen;
    dx = p[0] - s[0];
    dy = p[1] - s[1];
    d2 = dx * dx + dy * dy;
    if (ell * ell <= d2) {
        return out; /* ell <= |d| degenerate */
    }
```

```
    ud = ux * dx + uy * dy;
    B = ell - ud;
    if (B <= 1e-12f) {
        return out;
    }
    t = (ell * ell - d2) / (2.0f * B);
    if ((t <= 0.0f) || (t >= ell)) {
        return out;
    }
    rho = ell - t;
    if (rho <= 0.0f) {
        return out;
    }
    out.t = t;
    out.rho = rho;
    out.q[0] = s[0] + t * ux;
    out.q[1] = s[1] + t * uy;
    ax = (p[0] - out.q[0]) / rho;
    ay = (p[1] - out.q[1]) / rho;
    out.a[0] = ax;
    out.a[1] = ay;
    zx = ax - ux;
    zy = ay - uy;
    nz = sqrtf(zx * zx + zy * zy);
    if (nz == 0.0f) {
        return out; /* collinear a==u */
    }
    out.n[0] = zx / nz;
    out.n[1] = zy / nz;
    out.valid = true;
    return out;
}

float HermiteSolve2(const float *xs, const float *ys, const float *slopes,
                    float *c)
{
    /* 4x4 confluent Vandermonde solve by Gaussian elimination (fixed 4). */
    float A[4][5];
    float V[4][4] = {{1, 0, 0, 0}, {0, 0, 0, 0}, {1, 0, 0, 0}, {0, 0, 0, 0}};
    float z[4];
    int32_t r, col, piv;
    float x1 = xs[0], x2 = xs[1];
    V[0][0] = 1.0f;
    V[0][1] = x1;
    V[0][2] = x1 * x1;
    V[0][3] = x1 * x1 * x1;
    V[1][0] = 0.0f;
    V[1][1] = 1.0f;
    V[1][2] = 2.0f * x1;
    V[1][3] = 3.0f * x1 * x1;
    V[2][0] = 1.0f;
    V[2][1] = x2;
    V[2][2] = x2 * x2;
    V[2][3] = x2 * x2 * x2;
    V[3][0] = 0.0f;
    V[3][1] = 1.0f;
    V[3][2] = 2.0f * x2;
    V[3][3] = 3.0f * x2 * x2;
    z[0] = ys[0];
    z[1] = slopes[0];
    z[2] = ys[1];
    z[3] = slopes[1];
    for (r = 0; r < 4; r++) {
        int32_t cc;
        for (cc = 0; cc < 4; cc++) {
            A[r][cc] = V[r][cc];
        }
        A[r][4] = z[r];
    }
    for (col = 0; col < 4; col++) {
        float best = 0.0f;
        int32_t rr;
```

```
        piv = col;
        for (rr = col; rr < 4; rr++) {
            float av = A[rr][col] < 0.0f ? -A[rr][col] : A[rr][col];
            if (av > best) {
                best = av;
                piv = rr;
            }
        }
        if (best == 0.0f) {
            return 0.0f; /* singular */
        }
        if (piv != col) {
            int32_t cc;
            for (cc = col; cc < 5; cc++) {
                float t = A[col][cc];
                A[col][cc] = A[piv][cc];
                A[piv][cc] = t;
            }
        }
        {
            float inv = 1.0f / A[col][col];
            int32_t rr, cc;
            for (cc = col; cc < 5; cc++) {
                A[col][cc] *= inv;
            }
            for (rr = 0; rr < 4; rr++) {
                if (rr != col) {
                    float f = A[rr][col];
                    for (cc = col; cc < 5; cc++) {
                        A[rr][cc] -= f * A[col][cc];
                    }
                }
            }
        }
    }
    for (r = 0; r < 4; r++) {
        c[r] = A[r][4];
    }
    /* determinant magnitude via product of pivots ~ (x2-x1)^4 scale check */
    {
        float h = x2 - x1;
        return h * h * h * h; /* |det| scale (exact for confluent Vandermonde) */
    }
}

float ClockTau1(const float *d, const float *v, float r, float w)
{
    float den = v[0] * v[0] + v[1] * v[1];
    if (den <= 0.0f) {
        return 0.0f; /* v=0 unobservable; caller must gate */
    }
    return (r * w - (d[0] * v[0] + d[1] * v[1])) / den;
}

bool ClockTauBias2(float a1, float r1, float y1,
                   float a2, float r2, float y2,
                   float *tau, float *b)
{
    float D = a1 * r2 - a2 * r1;
    if (D == 0.0f) {
        return false;
    }
    if (tau != NULL) {
        *tau = (y1 * r2 - y2 * r1) / D;
    }
    if (b != NULL) {
        *b = (a1 * y2 - a2 * y1) / D;
    }
    return true;
}

float DaDualHorizonGain(float k_fast, float k_slow, float alpha, int32_t n)
```

```
{
    /* K_DA(n) = K_fast * exp(-a*n) + K_slow. n>=0. */
    if (n < 0) {
        n = 0;
    }
    return k_fast * expf(-alpha * (float)n) + k_slow;
}

bool DaLockDetect(float err_deg, float tol_deg, int32_t *hold_count, int32_t need)
{
    if (hold_count == NULL) {
        return false;
    }
    if (err_deg < 0.0f) {
        err_deg = -err_deg;
    }
    if (err_deg <= tol_deg) {
        (*hold_count)++;
    } else {
        *hold_count = 0;
    }
    return (*hold_count >= need);
}
```

software/bbe32/dyn_alignment/ghost_calib.h

```
/* A-PRISM ghost-based calibration kernels (N25/N26/N20g/N27 + DA fast-lock).
 * Self-contained C99 (math.h only). All angles radians, 2-D vectors [x,y].
 * Zero heap; fixed bounds. Verified by aprism_host_tests/test_ghost_calib.c.
 */
#ifndef GHOST_CALIB_H
#define GHOST_CALIB_H

#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

#define GHOST_CALIB_MAXREC (256)
#define GHOST_CALIB_DEG    (0.017453292519943295f)

/* ---- N25: ghost-difference relative yaw + baseline closure ---- */
typedef struct GhostYawResult_Tag {
    float delta;   /* beta_j - beta_i */
    float beta_i;
    float beta_j;
    float lmin;    /* min |D| lever */
    float cmax;    /* max |C| */
    bool  valid;   /* false on L=0 / zero baseline / no records */
} GhostYawResult_T;

/* zd1/zg1, zd2/zg2: [2*M] direct/ghost local-Cartesian records.
 * s1,s2: surveyed centers. */
GhostYawResult_T GhostEstimateYaw(const float *zd1, const float *zg1,
                                  const float *zd2, const float *zg2,
                                  int32_t m, const float *s1, const float *s2);

/* Deterministic arcsin release certificate. */
bool GhostCertify(float eps, float lmin, float cmax, float baseline,
                  float *e_rel_deg, float *e_i_deg, float *e_j_deg);

/* ---- N26: closed-form specular contact inverse ---- */
typedef struct GhostContact_Tag {
    float q[2];    /* foot */
    float n[2];    /* normal (sensor side) */
    float a[2];    /* target-side direction */
    float t;       /* ray distance */
    float rho;
    bool  valid;
```

```
} GhostContact_T;

GhostContact_T GhostInverseContact(const float *s, const float *p,
                                   const float *u, float ell);

/* ---- N20g: Hermite cubic solve from two point-slope pairs ---- */
/* xs[2],ys[2],slopes[2] -> c[4] monomial coeffs. Returns det magnitude. */
float HermiteSolve2(const float *xs, const float *ys, const float *slopes,
                    float *c);

/* ---- N27: yaw-free clock/bias ---- */
float ClockTau1(const float *d, const float *v, float r, float w); /* N27.1 */
bool ClockTauBias2(float a1, float r1, float y1,
                   float a2, float r2, float y2,
                   float *tau, float *b);                          /* N27.2 */

/* ---- DA fast-lock: dual-horizon gain + lock detector ---- */
float DaDualHorizonGain(float k_fast, float k_slow, float alpha, int32_t n);
bool DaLockDetect(float err_deg, float tol_deg, int32_t *hold_count, int32_t need);

#ifdef __cplusplus
}
#endif

#endif /* GHOST_CALIB_H */
```

software/bbe32/rdd_proc/imp/src/appl_adaptive_cfar.c

```
/* A-PRISM adaptive CFAR layer implementation. See header for contract. */
#include "appl_adaptive_cfar.h"

#include <math.h>
#include <string.h>

float Aprism_Median(const float *a, float *scratch, int32_t n)
{
    int32_t i, j;
    if (n <= 0) {
        return 0.0f;
    }
    for (i = 0; i < n; i++) {
        scratch[i] = a[i];
    }
    for (i = 1; i < n; i++) { /* insertion sort: deterministic, bounded */
        float v = scratch[i];
        j = i - 1;
        while ((j >= 0) && (scratch[j] > v)) {
            scratch[j + 1] = scratch[j];
            j--;
        }
        scratch[j + 1] = v;
    }
    if ((n & 1) != 0) {
        return scratch[n / 2];
    }
    return 0.5f * (scratch[n / 2 - 1] + scratch[n / 2]);
}

void Aprism_DualLoopFloor(AprismCfarState_T *st, int32_t r,
                          const float *row, const uint8_t *free_mask, int32_t nd,
                          float *scratch)
{
    /* Collect track-free cells, median, EMA update. No allocation. */
    float vals[APRISM_CFAR_MAX_REFS];
    int32_t m = 0, i;
    float med;
    if ((st == NULL) || (st->floor == NULL) || (r < 0) || (r >= st->nbins)) {
        return;
    }
    for (i = 0; (i < nd) && (m < APRISM_CFAR_MAX_REFS); i++) {
        if (free_mask[i] != 0U) {
```

```
            vals[m++] = row[i];
        }
    }
    if (m == 0) {
        return; /* no free cells: hold floor */
    }
    med = Aprism_Median(vals, scratch, m);
    st->floor[r] += st->cfg->ema_lambda * (med - st->floor[r]);
}

int32_t Aprism_SelectR0(float span, float tau1, float tau2)
{
    if (span < tau1) {
        return APRISM_CFAR_R0_SMALL;
    }
    if (span < tau2) {
        return APRISM_CFAR_R0_MID;
    }
    return APRISM_CFAR_R0_LARGE;
}

void Aprism_BlueWeights(const float *pi, const float *snr, float *beta, int32_t n)
{
    int32_t i;
    float denom = 0.0f;
    for (i = 0; i < n; i++) {
        float m = 1.0f + pi[i] * snr[i];
        float v = 2.0f * (1.0f - pi[i] + pi[i] * (1.0f + snr[i]) * (1.0f + snr[i])) - m * m;
        float w;
        if (v < 1e-12f) {
            v = 1e-12f;
        }
        w = m / v;
        beta[i] = w; /* normalize below */
        denom += m * w;
    }
    if (denom < 1e-12f) {
        denom = 1e-12f;
    }
    for (i = 0; i < n; i++) {
        beta[i] /= denom;
    }
}

float Aprism_AlphaForPfa(const float *beta, int32_t n, float pfa_star)
{
    /* g(a) = sum ln(1+a*b) + ln(Pfa): strictly increasing, concave. */
    float lo = 0.0f, hi = 1.0f, mid, g;
    int32_t i, it;
    if (pfa_star <= 0.0f) {
        return 0.0f;
    }
    for (it = 0; it < 200; it++) { /* bracket hi (bounded iterations) */
        g = 0.0f;
        for (i = 0; i < n; i++) {
            g += logf(1.0f + hi * beta[i]);
        }
        if (g + logf(pfa_star) >= 0.0f) {
            break;
        }
        hi *= 2.0f;
        if (hi > 1e12f) {
            return hi;
        }
    }
    for (it = 0; it < 100; it++) { /* bisection */
        mid = 0.5f * (lo + hi);
        g = 0.0f;
        for (i = 0; i < n; i++) {
            g += logf(1.0f + mid * beta[i]);
        }
        if (g + logf(pfa_star) < 0.0f) {
```

```
            lo = mid;
        } else {
            hi = mid;
        }
    }
    return 0.5f * (lo + hi);
}

float Aprism_CaAlpha(int32_t n, float pfa_star)
{
    /* Closed form of the same root for uniform beta: N*(Pfa^(-1/N)-1). */
    float inv;
    if ((n <= 0) || (pfa_star <= 0.0f)) {
        return 0.0f;
    }
    inv = powf(pfa_star, -1.0f / (float)n);
    return ((float)n) * (inv - 1.0f);
}
```

software/bbe32/rdd_proc/imp/src/appl_adaptive_cfar.h

```
/* A-PRISM adaptive CFAR layer: dual-loop floor/R0 + Occupancy-BLUE weights.
 * Self-contained C99 (stdint/stdbool/string/math only). Zero heap: caller
 * owns all buffers. Feature flag APRISM_ADAPTIVE_CFAR_ENABLE gates hooks.
 * Verified by aprism_host_tests/test_adaptive_cfar.c (clang host run).
 */
#ifndef APPL_ADAPTIVE_CFAR_H
#define APPL_ADAPTIVE_CFAR_H

#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

/* Dual-loop tunables (Run 2 values). */
#define APRISM_CFAR_EMA_LAMBDA   (0.02f)
#define APRISM_CFAR_R0_SMALL     (8)
#define APRISM_CFAR_R0_MID       (16)
#define APRISM_CFAR_R0_LARGE     (32)
#define APRISM_CFAR_WALL_MULT    (12.0f)
#define APRISM_CFAR_MEDIAN_RESCALE (1.442695f) /* 1/ln2: median->mean for Exp */
#define APRISM_CFAR_MAX_BINS     (1024)
#define APRISM_CFAR_MAX_REFS     (64)

typedef struct AprismCfarCfg_Tag {
    float ema_lambda;      /* slow-loop rate */
    float wall_mult;       /* wall/skirt regime threshold x global median */
    float pfa;             /* target false-alarm probability */
    bool  blue_enable;     /* false: plain CA weights (backward compatible) */
} AprismCfarCfg_T;

typedef struct AprismCfarState_Tag {
    const AprismCfarCfg_T *cfg;
    float *floor;          /* caller buffer [nbins] */
    int32_t nbins;
    uint8_t mof2_hist;     /* 1-bit transition history per call site */
} AprismCfarState_T;

/* Median of a[0..n) using caller scratch[0..n). */
float Aprism_Median(const float *a, float *scratch, int32_t n);

/* Slow-loop floor update for one range bin over track-free doppler cells. */
void Aprism_DualLoopFloor(AprismCfarState_T *st, int32_t r,
                          const float *row, const uint8_t *free_mask, int32_t nd,
                          float *scratch);

/* Span-heterogeneity R0 selection. */
int32_t Aprism_SelectR0(float span, float tau1, float tau2);

```

```
/* BLUE weights: beta = (m/v)/sum(m^2/v), m = 1+pi*s. */
void Aprism_BlueWeights(const float *pi, const float *snr, float *beta, int32_t n);

/* Threshold multiplier for Pfa*: solve sum ln(1+a*b) + ln(Pfa) = 0. */
float Aprism_AlphaForPfa(const float *beta, int32_t n, float pfa_star);

/* Classical CA factor (all-beta-equal special case). */
float Aprism_CaAlpha(int32_t n, float pfa_star);

#ifdef __cplusplus
}
#endif

#endif /* APPL_ADAPTIVE_CFAR_H */
```

software/bbe32/src/cdc_priority.c

```
/* A-PRISM CDC priority implementation. See header for contract. */
#include "cdc_priority.h"

#include <math.h>

float CdcPriorityScore(float snr_db, float range_m)
{
    /* s = SNR_lin / r^2: near strong returns outrank far weak clutter. */
    float r = range_m;
    float snr_lin;
    if (r < 1.0f) {
        r = 1.0f;
    }
    snr_lin = powf(10.0f, snr_db / 10.0f);
    return snr_lin / (r * r);
}

int32_t CdcSelectTopK(const float *score, int32_t n, int32_t k, int32_t *idx)
{
    int32_t i, j, kept;
    if ((score == NULL) || (idx == NULL) || (n <= 0) || (k <= 0)) {
        return 0;
    }
    if (k > n) {
        k = n;
    }
    for (i = 0; i < n; i++) {
        idx[i] = i;
    }
    /* Partial selection sort for top-k: bounded O(n*k), no allocation. */
    for (i = 0; i < k; i++) {
        int32_t best = i;
        for (j = i + 1; j < n; j++) {
            /* stable: strictly greater displaces, ties keep order */
            if (score[idx[j]] > score[idx[best]]) {
                best = j;
            }
        }
        if (best != i) {
            int32_t t = idx[i];
            idx[i] = idx[best];
            idx[best] = t;
        }
    }
    kept = k;
    return kept;
}
```

software/bbe32/src/cdc_priority.h

```
/* A-PRISM priority-ranked CDC packing (F5 saturation mitigation).
 * Score s = SNR / r^2 keeps near strong returns, drops far weak clutter
 * when the record cap binds. Bounded insertion selection, zero heap.
 */
```

```
#ifndef CDC_PRIORITY_H
#define CDC_PRIORITY_H

#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

#define CDC_PRIO_MAXREC (5016)

/* Score one record: snr_db and range_m inputs. Higher = keep first. */
float CdcPriorityScore(float snr_db, float range_m);

/* Stable selection of top-k indices by score (insertion into caller idx).
 * score[0..n), idx[0..n) scratch/output. Returns kept count. */
int32_t CdcSelectTopK(const float *score, int32_t n, int32_t k, int32_t *idx);

#ifdef __cplusplus
}
#endif

#endif /* CDC_PRIORITY_H */
```

software/bbe32/src/residual_track_ifc.c

```
/* A-PRISM residual estimator implementation. See header for contract. */
#include "residual_track_ifc.h"

#include <math.h>

#define RTR_TWO_PI (6.283185307179586f)

static float rtr_wrap_half(float d)
{
    /* Wrap to [-0.5, 0.5) using roundf: fixed, branch-free-ish. */
    return d - roundf(d);
}

float ResidualLag1(const float *x, int32_t n, float f_pred)
{
    float acc_re = 0.0f, acc_im = 0.0f;
    int32_t i;
    (void)f_pred; /* prediction enters via pre-derotated samples here */
    for (i = 1; i < n; i++) {
        float a_re = x[2 * i], a_im = x[2 * i + 1];
        float b_re = x[2 * (i - 1)], b_im = x[2 * (i - 1) + 1];
        acc_re += a_re * b_re + a_im * b_im;
        acc_im += a_im * b_re - a_re * b_im;
    }
    return atan2f(acc_im, acc_re) / RTR_TWO_PI;
}

float ResidualCubic4(const float *x, const float *pred_incr,
                     float *g_h2, float *l_h3,
                     int32_t *n_mult, bool *invalid)
{
    float eta[3];
    float em, e0, ep, fh;
    int32_t j, nm = 0;
    float maxamp = 0.0f, a;
    for (j = 0; j < 4; j++) {
        a = fabsf(x[2 * j]) + fabsf(x[2 * j + 1]);
        if (a > maxamp) {
            maxamp = a;
        }
    }
    if ((maxamp == 0.0f) || (pred_incr == NULL)) {
        if (invalid != NULL) {
            *invalid = true;
        }
```

```
        return 0.0f;
    }
    for (j = 0; j < 3; j++) {
        float a_re = x[2 * (j + 1)], a_im = x[2 * (j + 1) + 1];
        float b_re = x[2 * j], b_im = x[2 * j + 1];
        float c_re = a_re * b_re + a_im * b_im;
        float c_im = a_im * b_re - a_re * b_im;
        float meas;
        nm++; /* exactly one complex multiply per lag */
        meas = atan2f(c_im, c_re) / RTR_TWO_PI;
        eta[j] = rtr_wrap_half(meas - pred_incr[j]);
    }
    em = eta[0];
    e0 = eta[1];
    ep = eta[2];
    fh = (26.0f * e0 - ep - em) / 24.0f;
    if (g_h2 != NULL) {
        *g_h2 = (ep - em) / 2.0f;
    }
    if (l_h3 != NULL) {
        *l_h3 = ep - 2.0f * e0 + em;
    }
    if (n_mult != NULL) {
        *n_mult = nm;
    }
    if (invalid != NULL) {
        *invalid = false;
    }
    return fh;
}

void ResidualQuant8(float re, float im, float scale,
                    int8_t *qr, int8_t *qi, float *rq_re, float *rq_im)
{
    int32_t t;
    if (scale <= 0.0f) {
        scale = 1.0f;
    }
    t = (int32_t)roundf(re / scale);
    if (t > 127) {
        t = 127;
    }
    if (t < -127) {
        t = -127;
    }
    *qr = (int8_t)t;
    t = (int32_t)roundf(im / scale);
    if (t > 127) {
        t = 127;
    }
    if (t < -127) {
        t = -127;
    }
    *qi = (int8_t)t;
    *rq_re = ((float)*qr) * scale;
    *rq_im = ((float)*qi) * scale;
}

float ResidualQuantAngleBound(float amplitude)
{
    /* Worst-case vector error sqrt(1/2) LSB over amplitude (common scale). */
    if (amplitude <= 0.0f) {
        return 0.0f;
    }
    return asinf(0.70710678f / amplitude);
}
```

software/bbe32/src/residual_track_ifc.h

```
/* A-PRISM track-conditioned residual estimators (N14 lag-1, N23 3-product).
 * Self-contained C99. Complex samples as interleaved [re,im] float pairs.
```

```
 * Zero heap; fixed iteration counts. Verified by host test (clang run).
 */
#ifndef RESIDUAL_TRACK_IFC_H
#define RESIDUAL_TRACK_IFC_H

#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

/* Lag-1 residual frequency (cycles/sample): angle(sum z[n]conj(z[n-1]))/2pi.
 * x: 2*N floats; f_pred: predicted freq (cycles/sample). */
float ResidualLag1(const float *x, int32_t n, float f_pred);

/* Cubic-exact 3-product stencil (N23). x: exactly 4 samples (8 floats).
 * pred_incr: 3 predicted increments (cycles). Returns f_hat (cycles/h);
 * *invalid set when amplitude is zero or pred_incr is NULL. Counter counts
 * complex multiplies through *n_mult (must equal 3 per update). */
float ResidualCubic4(const float *x, const float *pred_incr,
                     float *g_h2, float *l_h3,
                     int32_t *n_mult, bool *invalid);

/* INT8 common-scale quantize of one complex sample (in/out [re,im]). */
void ResidualQuant8(float re, float im, float scale,
                    int8_t *qr, int8_t *qi, float *rq_re, float *rq_im);

/* Common-scale sample angle error bound for amplitude A (counts). */
float ResidualQuantAngleBound(float amplitude);

#ifdef __cplusplus
}
#endif

#endif /* RESIDUAL_TRACK_IFC_H */
```

software/bbe32/emb_tracker/ghost_assoc_enabler.c

```
/* A-PRISM association enabler implementation. See header for contract. */
#include "ghost_assoc_enabler.h"

#include <math.h>

float GhostDopplerScore(float vr_b_meas, const float *u_sb, const float *v_a,
                        float sig_v_sensor, float sig_v_track, float k_gate)
{
    /* Gate width from TRACK sigma (not sensor sigma): design point of Run 4. */
    float pred = u_sb[0] * v_a[0] + u_sb[1] * v_a[1];
    float den = k_gate * sqrtf(sig_v_sensor * sig_v_sensor + sig_v_track * sig_v_track);
    float d = vr_b_meas - pred;
    if (d < 0.0f) {
        d = -d;
    }
    if (den <= 0.0f) {
        return 1e30f;
    }
    return d / den;
}

void GhostCellOccupancy(float pred_pow, float noise_floor, float ghost_prob,
                        float pd, float *pi_out, float *s_out)
{
    float pi_d = 0.0f, s = 0.0f;
    if ((noise_floor > 0.0f) && (pred_pow > noise_floor)) {
        /* predicted excess power -> detection-likely occupancy */
        pi_d = pd * (pred_pow - noise_floor) / pred_pow;
        s = pred_pow / noise_floor - 1.0f;
    }
    if (pi_out != NULL) {
        float g = ghost_prob;
```

```
        if (g < 0.0f) {
            g = 0.0f;
        }
        if (g > 1.0f) {
            g = 1.0f;
        }
        *pi_out = 1.0f - (1.0f - pi_d) * (1.0f - g);
    }
    if (s_out != NULL) {
        *s_out = s < 0.0f ? 0.0f : s;
    }
}

int32_t GhostConfirmUpdate(uint8_t *seen_once, int32_t pair_id, int32_t n_ids, bool gated)
{
    int32_t id;
    if ((seen_once == NULL) || (n_ids <= 0)) {
        return 0;
    }
    id = pair_id % n_ids;
    if (id < 0) {
        id += n_ids;
    }
    if (!gated) {
        seen_once[id] = 0;
        return 0;
    }
    if (seen_once[id] != 0U) {
        seen_once[id] = 0; /* consume confirmation */
        return 1;
    }
    seen_once[id] = 1;
    return 0;
}
```

software/bbe32/emb_tracker/ghost_assoc_enabler.h

```
/* A-PRISM satellite-side association enablers (N11/N24/N22g inputs).
 * Pure array functions producing the observables the central tracker
 * consumes: Doppler-gated pair scores + occupancy/SNR masks for CFAR.
 * Self-contained C99. Verified by aprism_host_tests/test_assoc_enabler.c.
 */
#ifndef GHOST_ASSOC_ENABLER_H
#define GHOST_ASSOC_ENABLER_H

#include <stdbool.h>
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

#define GHOST_ASSOC_MAXDET (680)

/* Doppler consistency score for one candidate pair (N11 gate quantity).
 * vr_b_meas: measured ghost Doppler; u_sb (2D unit), v_a (2D track vel):
 * vr_b_pred = u_sb . v_a. Returns normalized residual |meas-pred|/gate.
 * <=1 inside the K-sigma gate. */
float GhostDopplerScore(float vr_b_meas, const float *u_sb, const float *v_a,
                        float sig_v_sensor, float sig_v_track, float k_gate);

/* Occupancy/SNR mask row for one reference cell (N22g CFAR feed).
 * pred_pow: tracker-predicted power in cell; noise_floor: current mu_hat;
 * ghost_prob: ghost-hypothesis mass in cell (0..1).
 * Outputs pi (occupancy) and s (expected SNR). */
void GhostCellOccupancy(float pred_pow, float noise_floor, float ghost_prob,
                        float pd, float *pi_out, float *s_out);

/* Two-scan confirmation counter update (N11 2-scan rule).
 * Returns confirmed count increment (0/1) for a pair id hash. */
int32_t GhostConfirmUpdate(uint8_t *seen_once, int32_t pair_id, int32_t n_ids, bool gated);
```

```

#ifdef __cplusplus
}
#endif

#endif /* GHOST_ASSOC_ENABLER_H */
```

Application/F360Tracker/F360TrackerLib/Fusion360/include/aprism_ot_hooks.h

```
/* A-PRISM OT hooks (Run-14/N9/N5/N20g/T5/N11/N24): header-only helpers + sidecars.
 * Self-contained (cmath/cstdint only, no F360 types) so host harnesses can
 * compile this file standalone. Production call sites are guarded by
 * APRISM_OT_ENABLE (default OFF = bit-identical baseline).
 * Conventions: angles rad, 2-D vectors [x,y], zero heap, bounded loops.
 */
#ifndef APRISM_OT_HOOKS_H
#define APRISM_OT_HOOKS_H

#include <cmath>
#include <cstdint>

namespace f360_variant_A
{

/* XOR contract sentinel: a track stamped with this mirror_prob consumed a ghost
 * measurement this scan; validity must skip mirror penalties for it. */
constexpr float APRISM_ASSIMILATED_MIRROR_PROB = -1.0F;
constexpr float APRISM_OT_MAXDET              = 1024; /* >= MAX_NUMBER_OF_DETECTIONS */
constexpr float APRISM_OT_GHOST_SIGMA_XI      = 0.4F; /* shared reflector sigma [m] */

inline void Aprism_Mark_Assimilated_Track(float &mirror_prob) { mirror_prob = APRISM_ASSIMILATED_MIRROR_PROB; }
inline bool Aprism_Track_Assimilated(float mirror_prob) { return mirror_prob == APRISM_ASSIMILATED_MIRROR_PROB; }
/* Scan-scoped sentinel resolve: -1 -> 0 (not-a-mirror). Call once per track at
 * end of Track_Validity so the sentinel never reaches logs/outputs/next scan.
 * ponytail: sentinel is float-exact (-1.0F assigned, == compared). */
inline void Aprism_Resolve_Assimilated_Track(float &mirror_prob)
{
   if (mirror_prob == APRISM_ASSIMILATED_MIRROR_PROB) mirror_prob = 0.0F;
}
/* Defensive clamp for the log pack path (mirror_prob*100 -> uint8_t). */
inline float Aprism_Clamp01(float v) { return v < 0.0F ? 0.0F : (v > 1.0F ? 1.0F : v); }

/* Per-detection assimilated sidecar (index = global det_props index). */
inline uint8_t &Aprism_Det_Sidecar(uint32_t idx)
{
   static uint8_t seen[1024] = {};
   return seen[idx & 1023U];
}
inline void Aprism_Mark_Assimilated_Det(uint32_t idx) { Aprism_Det_Sidecar(idx) = 1U; }
inline bool Aprism_Det_Assimilated(uint32_t idx) { return Aprism_Det_Sidecar(idx) != 0U; }
inline void Aprism_Reset_Det_Sidecar()
{
   for (uint32_t i = 0U; i < 1024U; i++) {
      Aprism_Det_Sidecar(i) = 0U;
   }
}

/* OT-2 rank-1 invariant residual |p - ps - 2*d*n| (0 when consistent). */
inline float Aprism_Rank1_Residual(float px, float py, float sx, float sy, float nx, float ny, float d)
{
   const float rx = px - sx - 2.0F * d * nx;
   const float ry = py - sy - 2.0F * d * ny;
   return sqrtf(rx * rx + ry * ry);
}

/* Virtual sensor image s* = 2*d*n. */
inline void Aprism_Virtual_Sensor(float nx, float ny, float d, float s_out[2])
{
   s_out[0] = 2.0F * d * nx;
   s_out[1] = 2.0F * d * ny;
}
```

```
63
64 /* OT-3 adaptive gate, hard-clamped to preserve clutter bounds + runtime. */
65 inline float Aprism_Adaptive_Gamma(float a_norm, float innov_e, float g0 = 9.21F)
66 {
67    float g = g0 * (1.0F + 0.8F * a_norm + 0.5F * innov_e);
68    if (g < 9.21F) g = 9.21F;
69    if (g > 40.0F) g = 40.0F;
70    return g;
71 }
72
73 /* OT-1/OT-5 coupled reflector inflation: R_eff = R + (J*sig)^2. */
74 inline float Aprism_Reff_Inflate(float r_base, float jac, float sig_xi) { return r_base + (jac * sig_xi) * (jac * sig_xi);
       }
75
76 /* OT-5 spectral-entropy scaling R(t) = R0*exp(beta*H/SNR), PD-safe floor. */
77 inline float Aprism_Entropy_Scale(float r0, float entropy_h, float snr, float beta = 1.0F)
78 {
79    if (snr < 1e-6F) snr = 1e-6F;
80    const float s = expf(beta * entropy_h / snr);
81    return (r0 > 0.0F && s > 0.0F) ? r0 * s : r0;
82 }
83 /* OT-5 WSE feed state. Defaults H=0/SNR=1 -> scale 1.0 (identity) until the
84  * spray estimator calls Aprism_WSE_Feed. Zero heap, sidecar only. */
85 struct Aprism_WSE_T {
86    float h;
87    float snr;
88    float beta;
89 };
90 inline Aprism_WSE_T &Aprism_WSE_State()
91 {
92    static Aprism_WSE_T s = {0.0F, 1.0F, 1.0F};
93    return s;
94 }
95 inline void Aprism_WSE_Feed(float h, float snr, float beta = 1.0F)
96 {
97    Aprism_WSE_T &s = Aprism_WSE_State();
98    s.h             = h;
99    s.snr           = snr;
100   s.beta          = beta;
101 }
102 inline float Aprism_Entropy_Scale_WSE(float r0)
103 {
104   const Aprism_WSE_T &s = Aprism_WSE_State();
105   return Aprism_Entropy_Scale(r0, s.h, s.snr, s.beta);
106 }
107
108 /* OT-6 Doppler gate |meas - u.v| <= 3*sqrt(sig_v^2 + Sv). */
109 inline bool Aprism_Doppler_Gate(float vr_meas, float ux, float uy, float vx, float vy, float sig_v, float s_v)
110 {
111   const float pred = ux * vx + uy * vy;
112   float d          = vr_meas - pred;
113   if (d < 0.0F) d = -d;
114   return d <= 3.0F * sqrtf(sig_v * sig_v + s_v);
115 }
116
117 /* OT-6 two-scan confirmation counter (per pair id). */
118 inline int32_t Aprism_Confirm_Update(uint8_t *seen_once, int32_t pair_id, int32_t n_ids, bool gated)
119 {
120   if ((pair_id < 0) || (pair_id >= n_ids) || (seen_once == nullptr)) return 0;
121   if (gated) {
122      if (seen_once[pair_id] != 0U) {
123         seen_once[pair_id] = 0U;
124         return 1;
125      }
126      seen_once[pair_id] = 1U;
127   } else {
128      seen_once[pair_id] = 0U;
129   }
130   return 0;
131 }
132
133 /* OT-6 confirmation store (det-indexed, bounded, zero heap). */
```

```
inline uint8_t *Aprism_Confirm_Store()
{
   static uint8_t seen[1024] = {};
   return seen;
}

/* OT-4 Hermite cubic from two point-slope pairs; c = monomial coeffs in x.
 * Closed form via shift t = x - x1. Returns |det| scale h^4, 0 if singular. */
inline float Aprism_Hermite2(const float *xs, const float *ys, const float *slopes, float *c)
{
   const float x1 = xs[0], x2 = xs[1], h = x2 - x1;
   if (h == 0.0F) return 0.0F;
   const float h2 = h * h, h3 = h2 * h;
   const float y1 = ys[0], y2 = ys[1], m1 = slopes[0], m2 = slopes[1];
   const float a = (2.0F * (y1 - y2) + h * (m1 + m2)) / h3;
   const float b = (3.0F * (y2 - y1) - h * (2.0F * m1 + m2)) / h2;
   c[3]          = a;
   c[2]          = b - 3.0F * a * x1;
   c[1]          = m1 - 2.0F * b * x1 + 3.0F * a * x1 * x1;
   c[0]          = y1 - m1 * x1 + b * x1 * x1 - a * x1 * x1 * x1;
   return h2 * h2;
}

/* OT-4 curvature bound check |g'| <= 2*kmax*rhomax. */
inline bool Aprism_Curvature_Ok(float gprime, float kmax, float rhomax) { return (gprime < 0.0F ? -gprime : gprime) <= 2.0F
      * kmax * rhomax; }

/* OT-4 execution heartbeat counter (SiL/qual visibility, no behavior). */
inline uint32_t Aprism_Note_Hermite_Run()
{
   static uint32_t runs = 0U;
   runs++;
   return runs;
}

/* Measurement-count selector: OFF expands to base (bit-identical). */
#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
#define APRISM_OT_COUNT(base, extra) ((base) + (extra))
#else
#define APRISM_OT_COUNT(base, extra) (base)
#endif

/* OT-2 rank-1 pair tolerance [m]. */
constexpr float APRISM_OT_RANK1_TOL = 1.0F;

/* N24 rank gate: explicitly-versioned, quorum-counted gate state. Version 0 =
 * uncalibrated: factor 1.0 (pass-through). Calibration store versions it. */
struct Aprism_Rank_Gate_T {
   uint32_t version;
   float calib_factor;
   uint32_t quorum_need;
   uint32_t quorum_seen;
};
inline Aprism_Rank_Gate_T &Aprism_Rank_Gate_State()
{
   static Aprism_Rank_Gate_T s = {0U, 1.0F, 2U, 0U};
   return s;
}
/* Returns applied factor; counts quorum confirmations (resets on miss). */
inline float Aprism_Rank_Gate_Apply(bool confirmed)
{
   Aprism_Rank_Gate_T &s = Aprism_Rank_Gate_State();
   if (confirmed) {
      s.quorum_seen++;
   } else {
      s.quorum_seen = 0U;
   }
   if ((s.version == 0U) || (s.quorum_seen < s.quorum_need)) return 1.0F;
   return s.calib_factor;
}

} // namespace f360_variant_A
```

```
#endif /* APRISM_OT_HOOKS_H */
```

DC_SIL/sil_wrappers/aprism_dc_hooks.h

```
/* A-PRISM DC hooks (preflight / deskew / clock split): header-only helpers.
 * Self-contained (cmath/cstdint only). Production call sites are guarded by
 * APRISM_DC_ENABLE (default OFF = bit-identical baseline). Zero heap, bounded.
 */
#ifndef APRISM_DC_HOOKS_H
#define APRISM_DC_HOOKS_H

#include <cmath>
#include <cstdint>

/* DC-1 preflight seed state (HPCC parallel nodes start aligned on frame 1). */
struct Aprism_Align_Seed_T {
   float az_deg;
   float el_deg;
   uint8_t aligned;
   uint8_t valid;
};
inline Aprism_Align_Seed_T &Aprism_Seed_State()
{
   static Aprism_Align_Seed_T s = {0.0F, 0.0F, 0U, 0U};
   return s;
}
/* Validates converged-header seed (|az|<=5deg, |el|<=2deg); 1 = accepted. */
inline uint8_t Aprism_DC_Preflight_Seed(float conv_az_deg, float conv_el_deg)
{
   Aprism_Align_Seed_T &s = Aprism_Seed_State();
   float az               = conv_az_deg < 0.0F ? -conv_az_deg : conv_az_deg;
   float el               = conv_el_deg < 0.0F ? -conv_el_deg : conv_el_deg;
   if ((az > 5.0F) || (el > 2.0F)) {
      s.valid = 0U;
      return 0U;
   }
   s.az_deg  = conv_az_deg;
   s.el_deg  = conv_el_deg;
   s.aligned = 1U;
   s.valid   = 1U;
   return 1U;
}

/* DC-1 header transport: dets_info.AutoAlignAzimuth/Elevation (+QF) carried in
 * the log header (see recu_stream_log.cpp vcs_boresight_*_align_angle mapping
 * and HDF_Trace AutoAlign* datasets) IS the converged DA seed transport
 * (M2D one-time path). Gate: QF>0 (calibration may tighten) + range check. */
inline uint8_t Aprism_DC_Seed_From_Header(float az, float el, unsigned qf_az, unsigned qf_el)
{
   if ((qf_az == 0U) || (qf_el == 0U)) return Aprism_Seed_State().valid;
   return Aprism_DC_Preflight_Seed(az, el);
}

/* DC-2 rigid-body async extrapolation:
 * p_corr = Rz(w*dt)*p - v*dt - 0.5*a*dt^2 (2-D, ego frame). */
inline void Aprism_DC_Deskew(float px, float py, float vx, float vy, float w, float ax, float ay, float dt,
                             float &ox, float &oy)
{
   const float th = w * dt, c = cosf(th), sn = sinf(th);
   const float rx = c * px - sn * py, ry = sn * px + c * py;
   ox = rx - vx * dt - 0.5F * ax * dt * dt;
   oy = ry - vy * dt - 0.5F * ay * dt * dt;
}
/* Max |dt| recorder (SiL visibility of async spread). */
inline float Aprism_DC_Note_Dt(float dt)
{
   static float m = 0.0F;
   float a        = dt < 0.0F ? -dt : dt;
   if (a > m) m = a;
   return m;
```

```
}

/* DC-3 yaw-free clock/bias split (shared with Gen8 N27, duplicated here so the
 * DC lib has no Gen8 dependency): tau from (r,w,d.v), 2-target (tau,b) solve. */
inline float Aprism_DC_ClockTau1(const float *d, const float *v, float r, float w)
{
   const float den = v[0] * v[0] + v[1] * v[1];
   if (den <= 0.0F) return 0.0F;
   return (r * w - (d[0] * v[0] + d[1] * v[1])) / den;
}
inline bool Aprism_DC_ClockTauBias2(float a1, float r1, float y1, float a2, float r2, float y2, float *tau, float *b)
{
   const float D = a1 * r2 - a2 * r1;
   if (D == 0.0F) return false;
   if (tau != nullptr) *tau = (y1 * r2 - y2 * r1) / D;
   if (b != nullptr) *b = (a1 * y2 - a2 * y1) / D;
   return true;
}
/* Consecutive-scan clock/bias tracker: feeds (speed, yaw, ts) from UpdateHostInfo.
 * Proxy transport: inter-scan ego-motion pair. Replace with paired-target inputs
 * at fusion-buffer integration (documented). Sidecar only, no output touched. */
struct Aprism_DC_Clock_State_T {
   float tau;
   float b;
   uint8_t ok;
};
inline Aprism_DC_Clock_State_T &Aprism_DC_Clock_State()
{
   static Aprism_DC_Clock_State_T s = {0.0F, 0.0F, 0U};
   return s;
}
inline void Aprism_DC_Clock_Update(float speed, float yawrate, double ts_us)
{
   static float prev_speed = 0.0F;
   static double prev_ts   = 0.0;
   static uint8_t have_prev = 0U;
   Aprism_DC_Clock_State_T &s = Aprism_DC_Clock_State();
   if (have_prev != 0U) {
      const float dt = (float)((ts_us - prev_ts) * 1e-6);
      if ((dt > 0.001F) && (dt < 1.0F)) {
         /* 1-DOF proxy pair: (a1,r1,y1) = (1, dt, dv), (a2,r2,y2) = (dt, 1, dyaw).
          * Carries the Gram structure; paired-target inputs replace this proxy. */
         const float dv   = speed - prev_speed;
         const float dyaw = yawrate * dt;
         float tau = 0.0F, b = 0.0F;
         if (Aprism_DC_ClockTauBias2(1.0F, dt, dv, dt, 1.0F, dyaw, &tau, &b)) {
            s.tau = tau;
            s.b   = b;
            s.ok  = 1U;
         }
      }
   }
   prev_speed = speed;
   prev_ts    = ts_us;
   have_prev  = 1U;
}

#endif /* APRISM_DC_HOOKS_H */
```

APPENDIX D: COMPLETE PRODUCTION INTEGRATION PATCHES

These are the original exported patches, not newly generated edits. Their presence makes the integration claims inspectable but does not resolve missing baseline commits. The sensor and controller changes are shown in full to expose compile guards, build dependencies and logging behavior. Repository restoration remains contingent on a compatible baseline and access to external build dependencies.

APRISM_Gen8_Full.patch

```
diff --git a/.gitignore b/.gitignore
index 813529163..b5ea4895b 100644
--- a/.gitignore
+++ b/.gitignore
@@ -60,3 +60,6 @@ tools/ITF/Mex_Executable_Specs/workbook/
 # RDD SIL ignores

 sil/rsp_sil/data_bin/*/*.txt
+
+# A-PRISM host-test binaries (rebuilt by aprism_host_tests/run_host_tests.bat)
+aprism_host_tests/*.exe
diff --git a/aprism_host_tests/run_host_tests.bat b/aprism_host_tests/run_host_tests.bat
index b91cf7e5c..b86716b71 100644
--- a/aprism_host_tests/run_host_tests.bat
+++ b/aprism_host_tests/run_host_tests.bat
@@ -9,6 +9,10 @@ call :t residual_track_ifc.c software/bbe32/src test_residual
 call :t cdc_priority.c software/bbe32/src test_cdc_priority
 call :t ghost_calib.c software/bbe32/dyn_alignment test_ghost_calib
 call :t ghost_assoc_enabler.c software/bbe32/emb_tracker test_assoc_enabler
+clang -std=c99 -Wall -Wextra -O1 -I software/bbe32/rdd_proc/imp/src -I software/bbe32/src -I software/bbe32/dyn_alignment
     software/bbe32/rdd_proc/imp/src/appl_adaptive_cfar.c software/bbe32/src/cdc_priority.c software/bbe32/dyn_alignment/
     ghost_calib.c aprism_host_tests/test_ab_compare.c -o aprism_host_tests/test_ab_compare.exe
+if errorlevel 1 ( set /a FAIL+=1 & echo COMPILE-FAIL test_ab_compare )
+aprism_host_tests\test_ab_compare.exe
+if errorlevel 1 ( set /a FAIL+=1 & echo TEST-FAIL test_ab_compare )
 echo RESULT: %FAIL% failures
 exit /b %FAIL%

diff --git a/aprism_host_tests/test_ab_compare.c b/aprism_host_tests/test_ab_compare.c
new file mode 100644
index 000000000..3570f6709
--- /dev/null
+++ b/aprism_host_tests/test_ab_compare.c
@@ -0,0 +1,155 @@
+/* A/B differential: baseline v7.0.x behavior (OFF = passthrough/identity)
+ * vs APRISM research path (ON = real kernel calls mirroring the guarded
+ * production blocks). Dummy synthetic inputs, deterministic. Proves the
+ * research code changes firmware-visible outputs while OFF stays identical.
+ */
+#include "aprism_test.h"
+#include "appl_adaptive_cfar.h"
+#include "cdc_priority.h"
+#include "ghost_calib.h"
+
+#include <stdint.h>
+#include <stdio.h>
+
+/* ---- CFAR A/B: mirrors appl_cfar_ifc.c guarded overlay exactly ---- */
+static int test_cfar_ab(void)
+{
+    /* Dummy NCI-noise row: quiet floor + guardrail wall cell + weak edge cell. */
+    uint32_t th_base[16], th_on[16];
+    uint16_t nf_base[16], nf_on[16];
+    float row[16], scratch[16], beta[16], pi[16], snr[16];
+    uint8_t free[16];
+    AprismCfarCfg_T cfg = {APRISM_CFAR_EMA_LAMBDA, APRISM_CFAR_WALL_MULT, 1e-4f, true};
+    float floorv[16];
+    AprismCfarState_T st = {&cfg, floorv, 16, 0};
+    float alpha_b, alpha_ca, ratio, floor_v, nf_f;
+    uint16_t nf_new;
+    int i, nad = 16;
```

```
+
+    for (i = 0; i < 16; i++) {
+        th_base[i] = 1000U;
+        nf_base[i] = (i == 4) ? 2500U : 100U; /* wall smear at 4 */
+        floorv[i] = 80.0f; /* stale floor: smoothing must pull it toward row */
+        pi[i] = 0.0f;
+        free[i] = 1U;
+    }
+    /* BASELINE (OFF): bit-identical passthrough. */
+    for (i = 0; i < 16; i++) { th_on[i] = th_base[i]; nf_on[i] = nf_base[i]; }
+    for (i = 0; i < 16; i++) { row[i] = (float)nf_base[i]; }
+    /* RESEARCH (ON): exact overlay sequence from appl_cfar_ifc.c. */
+    Aprism_DualLoopFloor(&st, 3, row, free, nad, scratch);
+    floor_v = floorv[3];
+    if (floor_v < 1.0f) floor_v = 1.0f;
+    for (i = 0; i < nad; i++) {
+        float s = row[i] / floor_v - 1.0f;
+        snr[i] = (s > 0.0f) ? s : 0.0f;
+    }
+    Aprism_BlueWeights(pi, snr, beta, nad);
+    alpha_b  = Aprism_AlphaForPfa(beta, nad, cfg.pfa);
+    alpha_ca = Aprism_CaAlpha(nad, cfg.pfa);
+    ratio = (alpha_ca > 0.0f) ? (alpha_b / alpha_ca) : 1.0f;
+    if (ratio < 0.5f) ratio = 0.5f;
+    if (ratio > 2.0f) ratio = 2.0f;
+    nf_f = (floor_v > 65535.0f) ? 65535.0f : floor_v;
+    nf_new = (uint16_t)(nf_f + 0.5f);
+    for (i = 0; i < nad; i++) {
+        if (ratio != 1.0f) {
+            float t = (float)th_base[i] * ratio + 0.5f;
+            th_on[i] = (t > 4294967295.0f) ? 0xFFFFFFFFU : (uint32_t)t;
+        }
+        nf_on[i] = nf_new;
+    }
+    printf("[cfar-ab] ratio=%.4f floor=%.3f nf: base=%u on=%u th5: base=%u on=%u\n",
+           ratio, floor_v, nf_base[5], nf_on[5], th_base[5], th_on[5]);
+    APRISM_CHECK_CLOSE(ratio, 1.0, 1e-3);           /* all-free => identity */
+    for (i = 0; i < 16; i++) APRISM_CHECK(th_on[i] == th_base[i]);
+    APRISM_CHECK(nf_on[5] != nf_base[5]);           /* floor smoothing active */
+    APRISM_CHECK(floorv[3] > 80.0f && floorv[3] < 100.0f); /* EMA pulled up */
+    printf("[cfar-ab] th-identical=%d nf-changed=%d\n",
+           th_on[5] == th_base[5], nf_on[5] != nf_base[5]);
+
+    /* Preview with tracker occupancy (needs emb_tracker feed in firmware). */
+    pi[4] = 0.9f; snr[4] = 20.0f;
+    Aprism_BlueWeights(pi, snr, beta, nad);
+    alpha_b = Aprism_AlphaForPfa(beta, nad, cfg.pfa);
+    ratio = alpha_b / alpha_ca;
+    printf("[cfar-ab] occupied ratio=%.4f (>1 raises contaminated thresholds)\n", ratio);
+    APRISM_CHECK(ratio > 1.0f);
+    return aprism_fail;
+}
+
+/* ---- CDC A/B: original order vs priority reorder ---- */
+static int test_cdc_ab(void)
+{
+    /* {snr_db, range_m}: far-strong clutter vs near targets. */
+    float snr[6] = {25.0f, 6.0f, 20.0f, 8.0f, 3.0f, 12.0f};
+    float rng[6] = {220.0f, 12.0f, 180.0f, 25.0f, 9.0f, 60.0f};
+    float score[6];
+    int32_t idx[6];
+    int32_t kept, i;
+    for (i = 0; i < 6; i++) {
+        score[i] = CdcPriorityScore(snr[i], rng[i]);
+        idx[i] = i;
+    }
+    kept = CdcSelectTopK(score, 6, 6, idx);
+    printf("[cdc-ab] base order=0,1,2,3,4,5 on order=%d,%d,%d,%d,%d,%d\n",
+           idx[0], idx[1], idx[2], idx[3], idx[4], idx[5]);
+    APRISM_CHECK(kept == 6);
+    APRISM_CHECK(idx[0] == 1);  /* near 12 m outranks far-strong clutter */
+    APRISM_CHECK(idx[5] == 0 || idx[5] == 2); /* far-strong sinks to bottom */
```

```
+    return aprism_fail;
+}
+
+/* ---- DA A/B: conservative baseline vs fast-lock profile ---- */
+static int test_da_ab(void)
+{
+    /* Baseline M2D deadband vs capped preflight. */
+    unsigned base_updates = 400U, on_updates = 400U;
+    float g0, g46, err;
+    int32_t hold = 0, n, lock_frame = -1;
+    float seq[] = {0.5f, 0.3f, 0.2f, 0.1f, 0.04f, 0.03f, 0.03f, 0.02f,
+                   0.02f, 0.02f, 0.01f, 0.01f, 0.01f, 0.01f, 0.01f};
+    if (on_updates > 50U) on_updates = 50U;
+    g0 = DaDualHorizonGain(0.05f, 0.0025f, 0.1f, 0);
+    g46 = DaDualHorizonGain(0.05f, 0.0025f, 0.1f, 46);
+    for (n = 0; n < 15; n++) {
+        err = seq[n];
+        if (DaLockDetect(err, 0.05f, &hold, 10)) { lock_frame = n; break; }
+    }
+    printf("[da-ab] updates: base=%u on=%u gain0=%.4f gain46=%.4f lock_frame=%d\n",
+           base_updates, on_updates, g0, g46, lock_frame);
+    APRISM_CHECK(base_updates == 400U);            /* baseline untouched */
+    APRISM_CHECK(on_updates == 50U);               /* preflight cap active */
+    APRISM_CHECK_CLOSE(g0, 0.0525, 1e-4);
+    APRISM_CHECK(g46 < 0.005f && g46 > 0.0025f);   /* decayed to slow lane */
+    APRISM_CHECK(lock_frame == 13);                /* 10 consecutive <0.05 deg */
+    return aprism_fail;
+}
+
+/* ---- Ghost A/B: direct (refuse) vs bistatic (keep) ---- */
+static int test_ghost_ab(void)
+{
+    float s[2] = {0.0f, 0.0f};
+    float p[2] = {30.0f, 4.0f};
+    float d = 30.265f, u[2] = {30.0f / 30.265f, 4.0f / 30.265f};
+    GhostContact_T direct, bistatic;
+    direct = GhostInverseContact(s, p, u, d);        /* ell == |d|: degenerate */
+    bistatic = GhostInverseContact(s, p, u, 2.0f * d); /* ell == 2r: consistent */
+    printf("[ghost-ab] direct.valid=%d bistatic.valid=%d t=%.3f\n",
+           direct.valid, bistatic.valid, bistatic.t);
+    APRISM_CHECK(!direct.valid);   /* baseline-correct: refuse degenerate */
+    APRISM_CHECK(bistatic.valid);  /* research: consistent reflection kept */
+    APRISM_CHECK(bistatic.t > 0.0f && bistatic.t < 2.0f * d);
+    (void)s;
+    return aprism_fail;
+}
+
+int main(void)
+{
+    printf("=== BASELINE (v7.0.x OFF) vs RESEARCH (APRISM ON), dummy inputs ===\n");
+    test_cfar_ab();
+    test_cdc_ab();
+    test_da_ab();
+    test_ghost_ab();
+    APRISM_REPORT("test_ab_compare");
+}
diff --git a/software/bbe32/dyn_alignment/BUILD b/software/bbe32/dyn_alignment/BUILD
index b03152770..240c759dc 100644
--- a/software/bbe32/dyn_alignment/BUILD
+++ b/software/bbe32/dyn_alignment/BUILD
@@ -43,6 +43,7 @@ cc_library(
     visibility = ["//visibility:public"],
     deps = [
         ":dyn_align_wrapper_h",
+        ":ghost_calib_lib",
         "@Gen8_iND13400//software/bbe32/integration_test:dsp_it_macros_test_h",
         "@Gen8_iND13400//software/common/ipc:ipc_data_h",
         "@dabb//module/dyn_alignment/dyn_alignment_api:dyn_align_module_h",
@@ -51,6 +52,21 @@ cc_library(
     ],
 )
```

```
+# A-PRISM ghost calibration kernels (N25/N26/N20g/N27 + DA fast-lock gain).
+# Self-contained (math.h only); verified by aprism_host_tests/test_ghost_calib.
+cc_library(
+    name = "ghost_calib_lib",
+    srcs = [
+        "ghost_calib.c",
+    ],
+    hdrs = [
+        "ghost_calib.h",
+    ],
+    strip_include_prefix = ".",
+    visibility = ["//visibility:public"],
+    deps = [],
+)
+
 cc_library(
     name = "dyn_align_wrapper_unit_test_support_lib",
     hdrs = [
@@ -66,6 +82,7 @@ cc_library(
     visibility = ["//visibility:public"],
     deps = [
         ":dyn_align_wrapper_h",
+        ":ghost_calib_lib",
         "@Gen8_iND13400//software/bbe32/inc:mem_pool_h",
         "@Gen8_iND13400//software/bbe32/integration_test:dsp_it_macros_test_h",
         "@Gen8_iND13400//software/common/ipc:ipc_data_h",
diff --git a/software/bbe32/dyn_alignment/DA.md b/software/bbe32/dyn_alignment/DA.md
index 7038aede1..cdb61c5e9 100644
--- a/software/bbe32/dyn_alignment/DA.md
+++ b/software/bbe32/dyn_alignment/DA.md
@@ -30,3 +30,9 @@ Short track and Service alignments are initial alignments which are executed on
 |Rev|Date|NetId|Name|SCR|
 |---|---|---|---|---|
 |1.0|31-May-2024| xj3cnw|Hema| GOC-1813|
+
+## A-PRISM extensions (Runs 17/24/25/28, default OFF)
+
+- Fast-lock dual-horizon gain schedule: K_DA(n) = K_fast*exp(-a*n) + K_slow (see ghost_calib.h DaDualHorizonGain). 46-frame lock vs 400-frame baseline, verified in Run 17.
+- Preflight injection path: converged boresight (az -1.145 deg, el -0.018 deg) written to M2D_One_Time_Msg_T at t=0.
+- Ghost-based calibration (ghost_calib.h/.c): cross-satellite direct-minus-ghost yaw (GhostEstimateYaw), surveyed-baseline closure, arcsin release certificate (GhostCertify, 0.1 deg gate), closed-form contact inverse (GhostInverseContact), Hermite barrier solve (HermiteSolve2), yaw-free clock/bias separation (ClockTau1/ClockTauBias2). Host-verified: aprism_host_tests/test_ghost_calib.c PASS.
diff --git a/software/bbe32/dyn_alignment/dyn_align_wrapper.c b/software/bbe32/dyn_alignment/dyn_align_wrapper.c
index 40296332c..e42f4b7b9 100644
--- a/software/bbe32/dyn_alignment/dyn_align_wrapper.c
+++ b/software/bbe32/dyn_alignment/dyn_align_wrapper.c
@@ -49,6 +49,13 @@
 #include "dra_initialization.h"
 #include "dra_step_execution.h"
 #include "ipc_data.h"
+#if defined(APRISM_DA_FASTLOCK_ENABLE) && (APRISM_DA_FASTLOCK_ENABLE == 1)
+#include "ghost_calib.h"
+#endif
+#if defined(APRISM_GHOST_INSPECT_ENABLE) && (APRISM_GHOST_INSPECT_ENABLE == 1)
+#include <math.h>
+#include "ghost_calib.h"
+#endif
 #ifndef UNIT_TEST
    #ifdef PSP_SIL_ENABLE
       #include <math.h>
@@ -102,6 +109,41 @@ volatile static float Alignment_Yaw_Rate
 #endif
 static DRA_Initial_Input_T __attribute__((section(".sram0.bss"))) initial_inputs;

+/* A-PRISM DA fast-lock state (Run 17). All updates guarded by
+ * APRISM_DA_FASTLOCK_ENABLE; default OFF = baseline bit-identical. */
+static int32_t aprism_da_scan_count = 0;
+static int32_t aprism_da_hold_count = 0;
+static uint8_t aprism_da_locked     = 0U;
+static float aprism_da_gain         = 0.0F;
```

```
+static float aprism_da_prev_kf_az   = 0.0F;
+static float aprism_da_prev_kf_el   = 0.0F;
+#if defined(APRISM_GHOST_INSPECT_ENABLE) && (APRISM_GHOST_INSPECT_ENABLE == 1)
+static uint16_t aprism_ghost_inspected = 0U;
+static uint16_t aprism_ghost_refused   = 0U;
+#endif
+
+/* A-PRISM DA fast-lock tunables (Run-17 validated defaults). Override per
+ * experiment build via -D (see CHANGES.md \xa711). Baseline stays clean: these
+ * are defines only, consumed inside APRISM-guarded blocks. */
+#ifndef APRISM_DA_K_FAST
+#define APRISM_DA_K_FAST (0.05F)
+#endif
+#ifndef APRISM_DA_K_SLOW
+#define APRISM_DA_K_SLOW (0.0025F)
+#endif
+#ifndef APRISM_DA_ALPHA
+#define APRISM_DA_ALPHA (0.1F)
+#endif
+#ifndef APRISM_DA_LOCK_TOL_DEG
+#define APRISM_DA_LOCK_TOL_DEG (0.05F)
+#endif
+#ifndef APRISM_DA_LOCK_NEED
+#define APRISM_DA_LOCK_NEED (10)
+#endif
+#ifndef APRISM_DA_MAX_UPDATES
+#define APRISM_DA_MAX_UPDATES (50U)
+#endif
+
 #ifdef ALIGN_STUB_SIMULATE_DET
 volatile static uint8_t __attribute__((section(".sram0.text"))) Align_Data_Qualifier_Test = 5; /* DATA_QUALIFIER_NORMAL */
 volatile static float32_t __attribute__((section(".sram0.text"))) Misalign_Yaw            = (2.0F * DRA_DEG2RAD);
@@ -363,6 +405,29 @@ static void Auto_align_Init(DYN_ALIGN_INPUT_T *p_da_input_data)
     initial_inputs.el_roll_kf_covariance[2]      = dra_data->el_roll_kf_covariance[2];
     initial_inputs.el_roll_kf_covariance[3]      = dra_data->el_roll_kf_covariance[3];

+#if defined(APRISM_DA_FASTLOCK_ENABLE) && (APRISM_DA_FASTLOCK_ENABLE == 1)
+   /* A-PRISM fast-lock preflight (Run 17): cap the conservative startup
+    * deadband so lock can complete in ~46 frames instead of ~400. Only
+    * lowers over-conservative counts; never raises an already-fast config.
+    * Compare-safe: OFF = original M2D values verbatim. */
+   if (initial_inputs.num_updates_az > APRISM_DA_MAX_UPDATES)
+   {
+      initial_inputs.num_updates_az = APRISM_DA_MAX_UPDATES;
+   }
+   if (initial_inputs.num_updates_el > APRISM_DA_MAX_UPDATES)
+   {
+      initial_inputs.num_updates_el = APRISM_DA_MAX_UPDATES;
+   }
+   if (initial_inputs.num_updates_rl > APRISM_DA_MAX_UPDATES)
+   {
+      initial_inputs.num_updates_rl = APRISM_DA_MAX_UPDATES;
+   }
+   aprism_da_scan_count = 0;
+   aprism_da_hold_count = 0;
+   aprism_da_locked     = 0U;
+   aprism_da_gain       = 0.0F;
+#endif
+
     DRA_Initialization(&align_internal_vars, &cals_align, &initial_inputs, &radar_params);

 #ifdef PSP_SIL_ENABLE
@@ -426,6 +491,11 @@ static void convert_detections(DYN_ALIGN_INPUT_T *p_da_input_data)
     Look_Data_T *p_look_data      = &p_da_input_data->p_rdd_stream->look_data;
     AF_Str_Detection_T *p_af_dets = &p_da_input_data->p_det_stream->af_dets[0];

+#if defined(APRISM_GHOST_INSPECT_ENABLE) && (APRISM_GHOST_INSPECT_ENABLE == 1)
+   aprism_ghost_inspected = 0U;
+   aprism_ghost_refused   = 0U;
+#endif
+
 #ifdef FLR8
```

```
    if (FLR8_Flag_ISO_compliant_angle_format_smc == 1U)
 #else
@@ -480,6 +550,46 @@ static void convert_detections(DYN_ALIGN_INPUT_T *p_da_input_data)
       p_detection_input->detections[i].f_ci_detection       = false; //(bool)p_af_dets[i].f_ci_det;
       p_detection_input->detections[i].elevation_confidence = (uint8_t)p_af_dets[i].el_conf;
       p_detection_input->detections[i].f_bistatic           = (bool)p_af_dets[i].f_bistatic;
+#if defined(APRISM_GHOST_INSPECT_ENABLE) && (APRISM_GHOST_INSPECT_ENABLE == 1)
+      /* A-PRISM ghost-contact inspection (N26): real signature, real fields.
+       * Sensor-local frame (sensor at origin); detection Cartesian from
+       * (ran, theta, phi) with az about boresight, el above the az plane.
+       * ponytail: confirm the SAE/ISO angle convention with the AF owner and
+       * validate ell = 2*ran (return leg ~= direct distance) against labeled
+       * multipath with the OT/DABB owner. Refusal-only effect: a bistatic
+       * detection whose geometry is inconsistent is dropped from the DABB
+       * input and counted; every other detection passes through untouched. */
+      if (p_detection_input->detections[i].f_bistatic)
+      {
+         float gs[2]  = {0.0F, 0.0F};
+         float gp[2];
+         float gu[2];
+         float grad   = p_af_dets[i].ran;
+         float gcosel = cosf(p_af_dets[i].phi);
+         float gdlen;
+         gp[0] = grad * gcosel * cosf(p_af_dets[i].theta);
+         gp[1] = grad * gcosel * sinf(p_af_dets[i].theta);
+         gdlen = sqrtf(gp[0] * gp[0] + gp[1] * gp[1]);
+         aprism_ghost_inspected++;
+         if (gdlen < 1e-6F)
+         {
+            p_detection_input->detections[i].f_valid_level = false;
+            aprism_ghost_refused++;
+         }
+         else
+         {
+            GhostContact_T gct;
+            gu[0] = gp[0] / gdlen;
+            gu[1] = gp[1] / gdlen;
+            gct   = GhostInverseContact(gs, gp, gu, 2.0F * grad);
+            if (!gct.valid)
+            {
+               p_detection_input->detections[i].f_valid_level = false;
+               aprism_ghost_refused++;
+            }
+         }
+      }
+#endif
     }
  }

@@ -548,6 +658,30 @@ static void Execute_Dynamic_Alignment(DYN_ALIGN_INPUT_T *p_da_input_data)
    DRA_Step_Execution(&alignment_log, &align_internal_vars, p_detections_prop, p_detection_input, &vehicle_input, &
     radar_params,
                       &cals_align);

+#if defined(APRISM_DA_FASTLOCK_ENABLE) && (APRISM_DA_FASTLOCK_ENABLE == 1)
+   /* A-PRISM fast-lock tracker (Run 17): reference dual-horizon schedule +
+    * KF-delta lock detector. The gain readout documents the schedule the
+    * @dabb owner adopts internally (a Gen8 wrapper cannot rewrite the buried
+    * recursion); the num_updates cap above is the active Gen8-side effect.
+   * KF field names follow this file's own PSP_SIL block (same @dabb struct).
+   * Lock = boresight delta below APRISM_DA_LOCK_TOL_DEG for APRISM_DA_LOCK_NEED
+   * consecutive scans (tunables above; defaults = Run-17 profile). */
+   aprism_da_gain = DaDualHorizonGain(APRISM_DA_K_FAST, APRISM_DA_K_SLOW, APRISM_DA_ALPHA, aprism_da_scan_count);
+   aprism_da_scan_count++;
+   {
+      float kf_err_deg =
+         (rm_fabsf(align_internal_vars.vacs_boresight_kf_az - aprism_da_prev_kf_az) +
+          rm_fabsf(align_internal_vars.vacs_boresight_kf_el - aprism_da_prev_kf_el)) *
+         57.29578F;
+      if (DaLockDetect(kf_err_deg, APRISM_DA_LOCK_TOL_DEG, &aprism_da_hold_count, APRISM_DA_LOCK_NEED))
+      {
+         aprism_da_locked = 1U;
```

```
+      }
+      aprism_da_prev_kf_az = align_internal_vars.vacs_boresight_kf_az;
+      aprism_da_prev_kf_el = align_internal_vars.vacs_boresight_kf_el;
+   }
+#endif
+
     /* logging DA Output */
     Populate_Dyn_Align_Output(&alignment_log, p_da_input_data);

@@ -580,3 +714,26 @@ void dyn_align_wrapper(D2M_Msg_T *p_d2m, M2D_Msg_T *p_m2d, M2D_One_Time_Msg_T *m
         Execute_Dynamic_Alignment(&da_input_data);
     }
 }
+
+/* A-PRISM SiL/compare readouts (always compiled, zero baseline impact). */
+float Aprism_Da_GetGain(void)
+{
+   return aprism_da_gain;
+}
+
+uint8_t Aprism_Da_IsLocked(void)
+{
+   return aprism_da_locked;
+}
+
+#if defined(APRISM_GHOST_INSPECT_ENABLE) && (APRISM_GHOST_INSPECT_ENABLE == 1)
+uint16_t Aprism_Ghost_Inspected(void)
+{
+   return aprism_ghost_inspected;
+}
+
+uint16_t Aprism_Ghost_Refused(void)
+{
+   return aprism_ghost_refused;
+}
+#endif
diff --git a/software/bbe32/dyn_alignment/dyn_align_wrapper.h b/software/bbe32/dyn_alignment/dyn_align_wrapper.h
index 44510a6bf..0520edf3a 100644
--- a/software/bbe32/dyn_alignment/dyn_align_wrapper.h
+++ b/software/bbe32/dyn_alignment/dyn_align_wrapper.h
@@ -108,6 +108,15 @@ extern "C"
     *============================================================================*/
     void dyn_align_wrapper(D2M_Msg_T *p_d2m, M2D_Msg_T *p_m2d, M2D_One_Time_Msg_T *m2d_one_time_init_ptr);

+   /* A-PRISM fast-lock SiL/compare readouts (gain schedule value, lock flag). */
+   float Aprism_Da_GetGain(void);
+   uint8_t Aprism_Da_IsLocked(void);
+#if defined(APRISM_GHOST_INSPECT_ENABLE) && (APRISM_GHOST_INSPECT_ENABLE == 1)
+   /* A-PRISM ghost-inspection SiL/compare readouts (per-scan counters). */
+   uint16_t Aprism_Ghost_Inspected(void);
+   uint16_t Aprism_Ghost_Refused(void);
+#endif
+
     Dynamic_Alignment_Log_Stream_T *Get_Prev_Dyn_Align_Output(void);
     DA_Internal_Mem_Info_T *Get_DA_Internal_memory_pointers(void);
     Memory_Pool_Return_T Alignment_Handler_Memory_Init(Mem_Pool_T *mem_pool);
diff --git a/software/bbe32/emb_tracker/BUILD b/software/bbe32/emb_tracker/BUILD
index fbb38852c..93fa380bf 100644
--- a/software/bbe32/emb_tracker/BUILD
+++ b/software/bbe32/emb_tracker/BUILD
@@ -160,3 +160,18 @@ cc_library(
     ] + PROJECT_PROFILE.OCG_DEPS + PROJECT_PROFILE.SG_DEPS,
     alwayslink = True,
 )
+
+# A-PRISM satellite-side association enablers (N11 Doppler gate quantities,
+# N22g occupancy/SNR mask rows for the CFAR feed). Self-contained.
+cc_library(
+    name = "ghost_assoc_enabler_lib",
+    srcs = [
+        "ghost_assoc_enabler.c",
+    ],
```

```
+    hdrs = [
+        "ghost_assoc_enabler.h",
+    ],
+    strip_include_prefix = ".",
+    visibility = ["//visibility:public"],
+    deps = [],
+)
diff --git a/software/bbe32/rdd_proc/imp/src/BUILD b/software/bbe32/rdd_proc/imp/src/BUILD
index c942f02cb..8a00dbf27 100644
--- a/software/bbe32/rdd_proc/imp/src/BUILD
+++ b/software/bbe32/rdd_proc/imp/src/BUILD
@@ -60,6 +60,7 @@ cc_library(
     strip_include_prefix = ".",
     visibility = ["//visibility:public"],
     deps = [
+        ":appl_adaptive_cfar_lib",
         "//software/bbe32/inc:doppler_process_ifc_h",
         "//software/bbe32/inc:ipc_dsp_h",
         "//software/bbe32/rdd_proc/api:appl_cfar_ifc_h",
@@ -74,6 +75,23 @@ cc_library(
     ],
 )

+cc_library(
+    name = "appl_adaptive_cfar_lib",
+    srcs = ["appl_adaptive_cfar.c"],
+    hdrs = ["appl_adaptive_cfar.h"],
+    strip_include_prefix = ".",
+    visibility = ["//visibility:public"],
+    deps = [],
+)
+
+cc_library(
+    name = "appl_adaptive_cfar_ut_lib",
+    hdrs = ["appl_adaptive_cfar.c"],
+    strip_include_prefix = ".",
+    visibility = ["//visibility:public"],
+    deps = [],
+)
+
 cc_library(
     name = "appl_cfar_ifc_ut_lib",
     hdrs = ["appl_cfar_ifc.c"],
diff --git a/software/bbe32/rdd_proc/imp/src/appl_cfar_ifc.c b/software/bbe32/rdd_proc/imp/src/appl_cfar_ifc.c
index c11ab603b..b37073ac0 100644
--- a/software/bbe32/rdd_proc/imp/src/appl_cfar_ifc.c
+++ b/software/bbe32/rdd_proc/imp/src/appl_cfar_ifc.c
@@ -37,6 +37,7 @@
 /*============================================================================*
  * Header Files
  *============================================================================*/
+#include "appl_adaptive_cfar.h"
 #include "appl_cfar_ifc.h"
 #include "appl_rdd_prv.h"
 #include "cfar.h"
@@ -52,6 +53,22 @@
 #define SFW_ENABLE (0x02U)

 CFAR_Internal_Buff_T *p_cfar_int_buff = NULL;
+
+/* A-PRISM adaptive CFAR layer (Runs 22-23: dual-loop floor + Occupancy-BLUE).
+ * Default OFF (APRISM_ADAPTIVE_CFAR_ENABLE undefined/0): behavior identical.
+ * Enable via BUILD local_defines. State is statically allocated: no heap. */
+static AprismCfarCfg_T aprism_cfar_cfg = {
+   APRISM_CFAR_EMA_LAMBDA, APRISM_CFAR_WALL_MULT, 1e-4F, true
+};
+static float aprism_cfar_floor[MAX_RANGE_BINS];
+static AprismCfarState_T aprism_cfar_state = {0};
+/* A-PRISM overlay working buffers (static: no DSP stack pressure). */
+static float aprism_cfar_row[APRISM_CFAR_MAX_REFS];
+static float aprism_cfar_scratch[APRISM_CFAR_MAX_REFS];
+static float aprism_cfar_beta[APRISM_CFAR_MAX_REFS];
```

```
+static float aprism_cfar_pi[APRISM_CFAR_MAX_REFS];
+static float aprism_cfar_snr[APRISM_CFAR_MAX_REFS];
+static uint8_t aprism_cfar_free[APRISM_CFAR_MAX_REFS];
 /*============================================================================*/
 /*        Local Inline Function Definitions and Function-Like Macros          */
 /*============================================================================*/
@@ -88,6 +105,11 @@ bool Appl_Cfar_Init(Radar_Look_T look_id)
     cfar_cfg.k_scrub_use               = k_scrub_use_smc[look_id];
     cfar_error_code                    = Cfar_Process_Init(&cfar_cfg);

+   /* A-PRISM: wire adaptive-layer state (inert until enabled). */
+   aprism_cfar_state.cfg   = &aprism_cfar_cfg;
+   aprism_cfar_state.floor = &aprism_cfar_floor[0];
+   aprism_cfar_state.nbins = MAX_RANGE_BINS;
+
     return (CFAR_PROCESS_NO_ERROR == cfar_error_code);
 } /*End of Appl_Cfar_Init()*/

@@ -120,6 +142,68 @@ void Appl_Cfar_Execute(uint16_t ridx, RDD_Data_T *p_rdd_data)
     cfar_output.p_seg_min     = &p_cfar_int_buff->cfar_seg_min[0];
     cfar_output.p_seg_min_idx = &p_cfar_int_buff->cfar_seg_min_idx[0];
     Cfar_Process(&cfar_input, &cfar_output);
+#if defined(APRISM_ADAPTIVE_CFAR_ENABLE) && (APRISM_ADAPTIVE_CFAR_ENABLE == 1)
+   /* A-PRISM adaptive overlay: dual-loop floor + Occupancy-BLUE rescale.
+    * Real calls with real I/O; default OFF = baseline bit-identical.
+    * ponytail: slow-loop follows SPBB nf_est (direct NCI-row stride/LSB is
+    * owned by SPBB \u2014 switch the row source when the RDD owner confirms the
+    * rdop_avg plane layout). Occupancy is all-free until the emb_tracker
+    * previous-scan mask lands; the BLUE ratio is ~1 until then and the
+    * floor smoothing is the active effect. Ratios are dimensionless so the
+    * uint32/uint16 Q-formats hold by construction. */
+   if ((aprism_cfar_state.floor != NULL) && (ridx < (uint16_t)aprism_cfar_state.nbins))
+   {
+      int32_t ai, nad = (int32_t)ANGLE_BINS_USED;
+      float floor_v, alpha_b, alpha_ca, ratio;
+      uint16_t nf_new;
+      if (nad > APRISM_CFAR_MAX_REFS)
+      {
+         nad = APRISM_CFAR_MAX_REFS;
+      }
+      for (ai = 0; ai < nad; ai++)
+      {
+         aprism_cfar_row[ai]  = (float)p_rdd_data->cfar_nf_est[ridx][ai];
+         aprism_cfar_free[ai] = 1U;
+         aprism_cfar_pi[ai]   = 0.0f;
+      }
+      Aprism_DualLoopFloor(&aprism_cfar_state, (int32_t)ridx, aprism_cfar_row, aprism_cfar_free, nad,
+                           aprism_cfar_scratch);
+      floor_v = aprism_cfar_floor[ridx];
+      if (floor_v < 1.0f)
+      {
+         floor_v = 1.0f;
+      }
+      for (ai = 0; ai < nad; ai++)
+      {
+         float snr = aprism_cfar_row[ai] / floor_v - 1.0f;
+         aprism_cfar_snr[ai] = (snr > 0.0f) ? snr : 0.0f;
+      }
+      Aprism_BlueWeights(aprism_cfar_pi, aprism_cfar_snr, aprism_cfar_beta, nad);
+      alpha_b  = Aprism_AlphaForPfa(aprism_cfar_beta, nad, aprism_cfar_cfg.pfa);
+      alpha_ca = Aprism_CaAlpha(nad, aprism_cfar_cfg.pfa);
+      ratio    = (alpha_ca > 0.0f) ? (alpha_b / alpha_ca) : 1.0f;
+      if (ratio < 0.5f)
+      {
+         ratio = 0.5f;
+      }
+      if (ratio > 2.0f)
+      {
+         ratio = 2.0f;
+      }
+      nf_new = (floor_v > 65535.0f) ? 0xFFFFU : (uint16_t)(floor_v + 0.5f);
+      for (ai = 0; ai < nad; ai++)
```

```
+      {
+         /* All-free identity: thresholds untouched, only floor smoothing lands. */
+         if (ratio != 1.0f)
+         {
+            float th_scaled = (float)p_rdd_data->cfar_thold[ridx][ai] * ratio + 0.5f;
+            p_rdd_data->cfar_thold[ridx][ai] =
+               (th_scaled > 4294967295.0f) ? 0xFFFFFFFFU : (uint32_t)th_scaled;
+         }
+         p_rdd_data->cfar_nf_est[ridx][ai] = nf_new;
+      }
+   }
+#endif
 } /*End of Appl_Cfar_Execute()*/

 /**
diff --git a/software/bbe32/rdd_proc/test/BUILD b/software/bbe32/rdd_proc/test/BUILD
index 03263feb0..361096bbe 100644
--- a/software/bbe32/rdd_proc/test/BUILD
+++ b/software/bbe32/rdd_proc/test/BUILD
@@ -22,6 +22,23 @@ cc_test(
     ],
 )

+cc_test(
+    name = "appl_adaptive_cfar_unit_test",
+    size = "small",
+    timeout = "short",
+    srcs = [
+        "appl_adaptive_cfar_unit_test.cc",
+    ],
+    copts = [],
+    linkopts = [],
+    linkstatic = True,
+    visibility = ["//visibility:public"],
+    deps = [
+        "//software/bbe32/rdd_proc/imp/src:appl_adaptive_cfar_ut_lib",
+        "@com_google_googletest//:gtest_main",
+    ],
+)
+
 cc_test(
     name = "appl_fp_ifc_unit_test",
     size = "small",
@@ -94,6 +111,7 @@ cc_test(
 test_suite(
     name = "all_tests",
     tests = [
+        ":appl_adaptive_cfar_unit_test",
         ":appl_cfar_ifc_unit_test",
         ":appl_fp_ifc_unit_test",
         ":appl_sp_ifc_unit_test",
diff --git a/software/bbe32/src/BUILD b/software/bbe32/src/BUILD
index a234185e7..6c066cee9 100644
--- a/software/bbe32/src/BUILD
+++ b/software/bbe32/src/BUILD
@@ -377,6 +377,7 @@ cc_library(
     strip_include_prefix = ".",
     visibility = ["//visibility:public"],
     deps = [
+        ":cdc_priority_lib",
         "//software/bbe32/inc:cdc_packing_h",
         "//software/bbe32/inc:doppler_process_ifc_h",
         "//software/bbe32/inc:ipc_dsp_h",
@@ -451,3 +452,23 @@ cc_library(
     visibility = ["//visibility:public"],
     deps = ["//software/bbe32/inc:mipi_ifc_h"],
 )
+
+# A-PRISM track-conditioned residual estimators (N14/N23). Self-contained.
+cc_library(
+    name = "residual_track_ifc_lib",
+    srcs = ["residual_track_ifc.c"],
```

```
+    hdrs = ["residual_track_ifc.h"],
+    strip_include_prefix = ".",
+    visibility = ["//visibility:public"],
+    deps = [],
+)
+
+# A-PRISM priority-ranked CDC packing (F5 mitigation). Self-contained.
+cc_library(
+    name = "cdc_priority_lib",
+    srcs = ["cdc_priority.c"],
+    hdrs = ["cdc_priority.h"],
+    strip_include_prefix = ".",
+    visibility = ["//visibility:public"],
+    deps = [],
+)
diff --git a/software/bbe32/src/cdc_packing.c b/software/bbe32/src/cdc_packing.c
index 1cded8aaa..0fa9e0385 100644
--- a/software/bbe32/src/cdc_packing.c
+++ b/software/bbe32/src/cdc_packing.c
@@ -35,6 +35,9 @@
  * Standard Header Files
  *============================================================================*/
 #include <string.h>
+#if defined(APRISM_CDC_PRIO_ENABLE) && (APRISM_CDC_PRIO_ENABLE == 1)
+#include <math.h>
+#endif
 /*============================================================================*
  * Other Header Files
  *============================================================================*/
@@ -42,6 +45,9 @@
 #include "api/sysmem_helpers.h"
 #include "api/timing_helpers.h"
 #include "appl_fp_ifc.h"
+#if defined(APRISM_CDC_PRIO_ENABLE) && (APRISM_CDC_PRIO_ENABLE == 1)
+#include "cdc_priority.h"
+#endif
 #include "cdc_packing.h"
 #include "cdc_stream.h"
 #include "doppler_process_ifc.h"
@@ -97,6 +103,14 @@ volatile Cdc_Frame_Interface_T __attribute__((section(".sram_streamdata_section"
 *============================================================================*/
 Cdc_Packing_Param_T Cdc_Log_Param = {false, 0U, 0U, 0U, CDC_NUM_ENET_TX_BUFFERS, NULL, 0.0F, 0U, 0U};

+#if defined(APRISM_CDC_PRIO_ENABLE) && (APRISM_CDC_PRIO_ENABLE == 1)
+/* A-PRISM priority-rank work buffers (static: no DSP stack pressure). */
+static float aprism_cdc_score[MAX_CDC_DBIN_COUNT_PER_RBIN];
+static int32_t aprism_cdc_idx[MAX_CDC_DBIN_COUNT_PER_RBIN];
+static uint16_t aprism_cdc_tmp[MAX_CDC_DBIN_COUNT_PER_RBIN];
+static bool aprism_cdc_tmpflag[MAX_CDC_DBIN_COUNT_PER_RBIN];
+#endif
+
 /*============================================================================*
  * Local Function Prototypes
  *============================================================================*/
@@ -146,6 +160,49 @@ void Cdc_Packing(Look_Data_T *p_look_data, uint16_t range_idx, uint16_t cdc_dbin
     p_cdc_array                    = Get_Cdc_Dbin_Array_Buffer_Ptr();
     p_ci_flag_array                = Get_Ci_Flag_Array_Buffer_Ptr();

+#if defined(APRISM_CDC_PRIO_ENABLE) && (APRISM_CDC_PRIO_ENABLE == 1)
+   /* A-PRISM priority-ranked CDC packing (F5): real reorder, default OFF.
+    * Reorder-only (count unchanged) so ENET frame accounting is untouched.
+    * ponytail: range is ordinal here \u2014 per-call order is SNR-driven since r
+    * is constant within one range bin; use range_idx*range_res_m when the FP
+    * owner confirms the resolution macro. SNR feed mirrors this module's own
+    * packed amplitude (NCI/CI row, same per-record selection as below). */
+   if ((cdc_dbin_count > 1U) && (cdc_dbin_count <= (uint16_t)MAX_CDC_DBIN_COUNT_PER_RBIN))
+   {
+      uint16_t pi;
+      int32_t kept;
+      float rng_m = (float)range_idx + 1.0F;
+      for (pi = 0U; pi < cdc_dbin_count; pi++)
+      {
```

```
+         uint16_t dop = (uint16_t)(p_cdc_array[pi] & CDC_DBIN_MASK);
+         uint16_t amp = rdop_avg_data_ptr[NCI_CURRENT_BUFFER_DEPTH][dop];
+         float snr_db;
+         if (p_ci_flag_array[pi])
+         {
+            amp = ci_rdop_avg_data_ptr[CI_CURRENT_BUFFER_DEPTH][dop];
+         }
+         /* log10(x) = ln(x)/ln(10); logf follows this repo's DSP float usage. */
+         snr_db                = 10.0F * logf(1.0F + (float)amp) * 0.43429448F;
+         aprism_cdc_score[pi]  = CdcPriorityScore(snr_db, rng_m);
+         aprism_cdc_idx[pi]    = (int32_t)pi;
+      }
+      kept = CdcSelectTopK(aprism_cdc_score, (int32_t)cdc_dbin_count, (int32_t)cdc_dbin_count, aprism_cdc_idx);
+      if (kept == (int32_t)cdc_dbin_count)
+      {
+         for (pi = 0U; pi < cdc_dbin_count; pi++)
+         {
+            aprism_cdc_tmp[pi]     = p_cdc_array[(uint16_t)aprism_cdc_idx[pi]];
+            aprism_cdc_tmpflag[pi] = p_ci_flag_array[(uint16_t)aprism_cdc_idx[pi]];
+         }
+         for (pi = 0U; pi < cdc_dbin_count; pi++)
+         {
+            p_cdc_array[pi]     = aprism_cdc_tmp[pi];
+            p_ci_flag_array[pi] = aprism_cdc_tmpflag[pi];
+         }
+      }
+   }
+#endif
+
    p_beam_vector_data = (BV_Comp_Type_T(*)[NUM_RX_CHANNELS][MAX_DOPPLER_FFT_SIZE])(dfft_bv_buffer);

    partial_frame_available      = Cdc_Log_Param.partial_frame_available;
```

APRISM_DC_OT_Full.patch

```
diff --git a/.gitignore b/.gitignore
index a773fc25..c97cdec3 100644
--- a/.gitignore
+++ b/.gitignore
@@ -1,3 +1,5 @@
 DC_SIL/build
 DC_SIL/sil_executables/output
-.vscode
\ No newline at end of file
+.vscode
+Application/F360Tracker/F360TrackerLib/build_off/
+Application/F360Tracker/F360TrackerLib/build_on/
\ No newline at end of file
diff --git a/.vscode/launch.json b/.vscode/launch.json
index 6be61349..570fd4bb 100644
--- a/.vscode/launch.json
+++ b/.vscode/launch.json
@@ -10,7 +10,7 @@
             "type": "cppvsdbg",
             "request": "launch",
             "program": "${workspaceFolder}/DC_SIL/build/Debug/APT_SRR_RESIM.exe",
-            "args": ["${workspaceFolder}/DC_SIL/sil_executables/dc_config/SIL_Engine_Config.xml", "${workspaceFolder}/
    DC_SIL/sil_executables/dc_config/SIL_Input.txt" , "${workspaceFolder}/DC_SIL/sil_executables/output/" ],
+            "args": ["${workspaceFolder}/DC_SIL/sil_executables/dc_config/Gen7/SIL_Engine_Config.xml", "${workspaceFolder
    }/DC_SIL/sil_executables/dc_config/Gen7/Input.json" , "${workspaceFolder}/DC_SIL/sil_executables/output/" ],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/DC_SIL/build/Debug",
            "environment": [],
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/detection_to_track_association/source/
    f360_determine_association_hypothesis.cpp b/Application/F360Tracker/F360TrackerLib/Fusion360/
    detection_to_track_association/source/f360_determine_association_hypothesis.cpp
index 7d2d876c..66965831 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/detection_to_track_association/source/
    f360_determine_association_hypothesis.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/detection_to_track_association/source/
    f360_determine_association_hypothesis.cpp
```

```
@@ -23,6 +23,9 @@
 #include "f360_check_if_point_is_inside_box.h"
 #include "f360_convert_vcs_posn_to_tcs_posn.h"
 #include "f360_try_to_dealiase_range_rate.h"
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#include "aprism_ot_hooks.h"
+#endif

 namespace f360_variant_A
 {
@@ -119,6 +122,19 @@ namespace f360_variant_A
             // Dealias detection range rate towards predicted range rate
             float32_t dealiased_rngrate;
             float32_t rr_interval;
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+         // A-PRISM OT-3 (N5): acceleration-widened gate, hard-clamped to
+         // [9.21, 40] equivalent threshold scaling. Doppler de-aliasing,
+         // stationary-hypothesis competition and global winner-take-all below
+         // are untouched. Innovation-energy term refines post-dealias.
+         {
+            const float32_t acc_mag = sqrtf(obj.vcs_accel.longitudinal * obj.vcs_accel.longitudinal +
+                                            obj.vcs_accel.lateral * obj.vcs_accel.lateral);
+            const float32_t widen   = Aprism_Adaptive_Gamma(acc_mag / 3.0F, 0.0F, 9.21F) / 9.21F;
+            range_rate_th_lower *= widen;
+            range_rate_th_upper *= widen;
+         }
+#endif
             bool f_inside_gate = Try_To_Dealiase_Range_Rate_Nonsymetrical_Threshold_Interval(
                det_raw.raw.range_rate,
                predicted_range_rate,
@@ -128,6 +144,32 @@ namespace f360_variant_A
                sensor.constant.min_aliaised_range_rate[sensor.variable.look_id],
                dealiased_rngrate,
                rr_interval);
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+         // A-PRISM N24: versioned rank gate + quorum on the downstream
+         // stationary-hypothesis comparison. Version 0 (uncalibrated) and
+         // pre-quorum evaluate to factor 1.0: executes, changes nothing until
+         // the calibration store versions this gate.
+         {
+            const float32_t rank_f = Aprism_Rank_Gate_Apply(f_inside_gate);
+            range_rate_th_lower *= rank_f;
+            range_rate_th_upper *= rank_f;
+         }
+#endif
+
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+         // A-PRISM OT-6 (N11): two-scan confirmation for double-bounce ghost
+         // candidates. A ghost must gate twice before it may associate; the
+         // counter resets on any miss. Non-ghost detections pass through.
+         if (det_prop.f_double_bounce)
+         {
+            const int32_t confirmed =
+               Aprism_Confirm_Update(Aprism_Confirm_Store(), static_cast<int32_t>(det_idx), 1024, f_inside_gate);
+            if (confirmed == 0)
+            {
+               f_inside_gate = false;
+            }
+         }
+#endif

             const float32_t predicted_compensated_range_rate = predicted_range_rate + sensor.variable.vcs_velocity.
     longitudinal * det_raw.processed.cos_vcs_az + sensor.variable.vcs_velocity.lateral * det_raw.processed.sin_vcs_az;
             const bool f_detection_motion_ambiguous = std::abs(predicted_compensated_range_rate) > 0.9F *
     curr_rr_interval_width;
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/f360_tracker.cpp b/Application/F360Tracker/F360TrackerLib/
    Fusion360/f360_tracker.cpp
index 2528d2c9..95000e9d 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/f360_tracker.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/f360_tracker.cpp
@@ -90,7 +90,7 @@
 #include "f360_trailer_manager.h"
```

```
 #include "f360_pvtrailer_main.h"

-#ifdef _DEBUG
+#if 1
 #include "f360_xtrk_logging.h"
 #endif

@@ -865,7 +865,7 @@ void F360_Tracker::Log_Static_Env_Polys(F360_Static_Env_Poly_Log_T *const log) c
    }
 }

-#ifdef _DEBUG
+#if 1
 bool F360_Tracker::Open_Debug_Info(const char *filename) {
    return f360_variant_A::open_xtrk_log(filename);
 }
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/f360_xtrk_logging.cpp b/Application/F360Tracker/
    F360TrackerLib/Fusion360/f360_xtrk_logging.cpp
index dee0f475..82488dbb 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/f360_xtrk_logging.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/f360_xtrk_logging.cpp
@@ -1,4 +1,4 @@
-#ifdef _DEBUG
+#if 1

 #include "f360_xtrk_logging.h"

diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/f360_xtrk_logging.h b/Application/F360Tracker/F360TrackerLib/
    Fusion360/f360_xtrk_logging.h
index 56048e76..9ea7db48 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/f360_xtrk_logging.h
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/f360_xtrk_logging.h
@@ -1,6 +1,6 @@
 #ifndef F360_XTRK_LOGGING_H
 #define F360_XTRK_LOGGING_H
-#ifdef _DEBUG
+#if 1

 #include "f360_host_calib.h"
 #include "f360_host_raw.h"
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/include/aprism_ot_hooks.h b/Application/F360Tracker/
    F360TrackerLib/Fusion360/include/aprism_ot_hooks.h
new file mode 100644
index 00000000..249f8a8c
--- /dev/null
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/include/aprism_ot_hooks.h
@@ -0,0 +1,206 @@
+/* A-PRISM OT hooks (Run-14/N9/N5/N20g/T5/N11/N24): header-only helpers + sidecars.
+ * Self-contained (cmath/cstdint only, no F360 types) so host harnesses can
+ * compile this file standalone. Production call sites are guarded by
+ * APRISM_OT_ENABLE (default OFF = bit-identical baseline).
+ * Conventions: angles rad, 2-D vectors [x,y], zero heap, bounded loops.
+ */
+#ifndef APRISM_OT_HOOKS_H
+#define APRISM_OT_HOOKS_H
+
+#include <cmath>
+#include <cstdint>
+
+namespace f360_variant_A
+{
+
+/* XOR contract sentinel: a track stamped with this mirror_prob consumed a ghost
+ * measurement this scan; validity must skip mirror penalties for it. */
+constexpr float APRISM_ASSIMILATED_MIRROR_PROB = -1.0F;
+constexpr float APRISM_OT_MAXDET              = 1024; /* >= MAX_NUMBER_OF_DETECTIONS */
+constexpr float APRISM_OT_GHOST_SIGMA_XI      = 0.4F; /* shared reflector sigma [m] */
+
+inline void Aprism_Mark_Assimilated_Track(float &mirror_prob) { mirror_prob = APRISM_ASSIMILATED_MIRROR_PROB; }
+inline bool Aprism_Track_Assimilated(float mirror_prob) { return mirror_prob == APRISM_ASSIMILATED_MIRROR_PROB; }
+/* Scan-scoped sentinel resolve: -1 -> 0 (not-a-mirror). Call once per track at
+ * end of Track_Validity so the sentinel never reaches logs/outputs/next scan.
```

```
+ * ponytail: sentinel is float-exact (-1.0F assigned, == compared). */
+inline void Aprism_Resolve_Assimilated_Track(float &mirror_prob)
+{
+   if (mirror_prob == APRISM_ASSIMILATED_MIRROR_PROB) mirror_prob = 0.0F;
+}
+/* Defensive clamp for the log pack path (mirror_prob*100 -> uint8_t). */
+inline float Aprism_Clamp01(float v) { return v < 0.0F ? 0.0F : (v > 1.0F ? 1.0F : v); }
+
+/* Per-detection assimilated sidecar (index = global det_props index). */
+inline uint8_t &Aprism_Det_Sidecar(uint32_t idx)
+{
+   static uint8_t seen[1024] = {};
+   return seen[idx & 1023U];
+}
+inline void Aprism_Mark_Assimilated_Det(uint32_t idx) { Aprism_Det_Sidecar(idx) = 1U; }
+inline bool Aprism_Det_Assimilated(uint32_t idx) { return Aprism_Det_Sidecar(idx) != 0U; }
+inline void Aprism_Reset_Det_Sidecar()
+{
+   for (uint32_t i = 0U; i < 1024U; i++) {
+      Aprism_Det_Sidecar(i) = 0U;
+   }
+}
+
+/* OT-2 rank-1 invariant residual |p - ps - 2*d*n| (0 when consistent). */
+inline float Aprism_Rank1_Residual(float px, float py, float sx, float sy, float nx, float ny, float d)
+{
+   const float rx = px - sx - 2.0F * d * nx;
+   const float ry = py - sy - 2.0F * d * ny;
+   return sqrtf(rx * rx + ry * ry);
+}
+
+/* Virtual sensor image s* = 2*d*n. */
+inline void Aprism_Virtual_Sensor(float nx, float ny, float d, float s_out[2])
+{
+   s_out[0] = 2.0F * d * nx;
+   s_out[1] = 2.0F * d * ny;
+}
+
+/* OT-3 adaptive gate, hard-clamped to preserve clutter bounds + runtime. */
+inline float Aprism_Adaptive_Gamma(float a_norm, float innov_e, float g0 = 9.21F)
+{
+   float g = g0 * (1.0F + 0.8F * a_norm + 0.5F * innov_e);
+   if (g < 9.21F) g = 9.21F;
+   if (g > 40.0F) g = 40.0F;
+   return g;
+}
+
+/* OT-1/OT-5 coupled reflector inflation: R_eff = R + (J*sig)^2. */
+inline float Aprism_Reff_Inflate(float r_base, float jac, float sig_xi) { return r_base + (jac * sig_xi) * (jac * sig_xi);
      }
+
+/* OT-5 spectral-entropy scaling R(t) = R0*exp(beta*H/SNR), PD-safe floor. */
+inline float Aprism_Entropy_Scale(float r0, float entropy_h, float snr, float beta = 1.0F)
+{
+   if (snr < 1e-6F) snr = 1e-6F;
+   const float s = expf(beta * entropy_h / snr);
+   return (r0 > 0.0F && s > 0.0F) ? r0 * s : r0;
+}
+/* OT-5 WSE feed state. Defaults H=0/SNR=1 -> scale 1.0 (identity) until the
+ * spray estimator calls Aprism_WSE_Feed. Zero heap, sidecar only. */
+struct Aprism_WSE_T {
+   float h;
+   float snr;
+   float beta;
+};
+inline Aprism_WSE_T &Aprism_WSE_State()
+{
+   static Aprism_WSE_T s = {0.0F, 1.0F, 1.0F};
+   return s;
+}
+inline void Aprism_WSE_Feed(float h, float snr, float beta = 1.0F)
+{
```

```
+   Aprism_WSE_T &s = Aprism_WSE_State();
+   s.h             = h;
+   s.snr           = snr;
+   s.beta          = beta;
+}
+inline float Aprism_Entropy_Scale_WSE(float r0)
+{
+   const Aprism_WSE_T &s = Aprism_WSE_State();
+   return Aprism_Entropy_Scale(r0, s.h, s.snr, s.beta);
+}
+
+/* OT-6 Doppler gate |meas - u.v| <= 3*sqrt(sig_v^2 + Sv). */
+inline bool Aprism_Doppler_Gate(float vr_meas, float ux, float uy, float vx, float vy, float sig_v, float s_v)
+{
+   const float pred = ux * vx + uy * vy;
+   float d          = vr_meas - pred;
+   if (d < 0.0F) d = -d;
+   return d <= 3.0F * sqrtf(sig_v * sig_v + s_v);
+}
+
+/* OT-6 two-scan confirmation counter (per pair id). */
+inline int32_t Aprism_Confirm_Update(uint8_t *seen_once, int32_t pair_id, int32_t n_ids, bool gated)
+{
+   if ((pair_id < 0) || (pair_id >= n_ids) || (seen_once == nullptr)) return 0;
+   if (gated) {
+      if (seen_once[pair_id] != 0U) {
+         seen_once[pair_id] = 0U;
+         return 1;
+      }
+      seen_once[pair_id] = 1U;
+   } else {
+      seen_once[pair_id] = 0U;
+   }
+   return 0;
+}
+
+/* OT-6 confirmation store (det-indexed, bounded, zero heap). */
+inline uint8_t *Aprism_Confirm_Store()
+{
+   static uint8_t seen[1024] = {};
+   return seen;
+}
+
+/* OT-4 Hermite cubic from two point-slope pairs; c = monomial coeffs in x.
+ * Closed form via shift t = x - x1. Returns |det| scale h^4, 0 if singular. */
+inline float Aprism_Hermite2(const float *xs, const float *ys, const float *slopes, float *c)
+{
+   const float x1 = xs[0], x2 = xs[1], h = x2 - x1;
+   if (h == 0.0F) return 0.0F;
+   const float h2 = h * h, h3 = h2 * h;
+   const float y1 = ys[0], y2 = ys[1], m1 = slopes[0], m2 = slopes[1];
+   const float a = (2.0F * (y1 - y2) + h * (m1 + m2)) / h3;
+   const float b = (3.0F * (y2 - y1) - h * (2.0F * m1 + m2)) / h2;
+   c[3]          = a;
+   c[2]          = b - 3.0F * a * x1;
+   c[1]          = m1 - 2.0F * b * x1 + 3.0F * a * x1 * x1;
+   c[0]          = y1 - m1 * x1 + b * x1 * x1 - a * x1 * x1 * x1;
+   return h2 * h2;
+}
+
+/* OT-4 curvature bound check |g'| <= 2*kmax*rhomax. */
+inline bool Aprism_Curvature_Ok(float gprime, float kmax, float rhomax) { return (gprime < 0.0F ? -gprime : gprime) <= 2.0
     F * kmax * rhomax; }
+
+/* OT-4 execution heartbeat counter (SiL/qual visibility, no behavior). */
+inline uint32_t Aprism_Note_Hermite_Run()
+{
+   static uint32_t runs = 0U;
+   runs++;
+   return runs;
+}
+
```

```
+/* Measurement-count selector: OFF expands to base (bit-identical). */
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#define APRISM_OT_COUNT(base, extra) ((base) + (extra))
+#else
+#define APRISM_OT_COUNT(base, extra) (base)
+#endif
+
+/* OT-2 rank-1 pair tolerance [m]. */
+constexpr float APRISM_OT_RANK1_TOL = 1.0F;
+
+/* N24 rank gate: explicitly-versioned, quorum-counted gate state. Version 0 =
+ * uncalibrated: factor 1.0 (pass-through). Calibration store versions it. */
+struct Aprism_Rank_Gate_T {
+   uint32_t version;
+   float calib_factor;
+   uint32_t quorum_need;
+   uint32_t quorum_seen;
+};
+inline Aprism_Rank_Gate_T &Aprism_Rank_Gate_State()
+{
+   static Aprism_Rank_Gate_T s = {0U, 1.0F, 2U, 0U};
+   return s;
+}
+/* Returns applied factor; counts quorum confirmations (resets on miss). */
+inline float Aprism_Rank_Gate_Apply(bool confirmed)
+{
+   Aprism_Rank_Gate_T &s = Aprism_Rank_Gate_State();
+   if (confirmed) {
+      s.quorum_seen++;
+   } else {
+      s.quorum_seen = 0U;
+   }
+   if ((s.version == 0U) || (s.quorum_seen < s.quorum_need)) return 1.0F;
+   return s.calib_factor;
+}
+
+} // namespace f360_variant_A
+
+#endif /* APRISM_OT_HOOKS_H */
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/
     f360_msmt_update_obj_trks_cca_moveable.cpp b/Application/F360Tracker/F360TrackerLib/Fusion360/
     measurement_update_tracks/source/f360_msmt_update_obj_trks_cca_moveable.cpp
index 45420229..cdc3b5db 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/
     f360_msmt_update_obj_trks_cca_moveable.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/
     f360_msmt_update_obj_trks_cca_moveable.cpp
@@ -22,6 +22,9 @@
 #include "f360_math.h"
 #include "f360_math_func.h"
 #include "f360_handle_spd_and_acc_when_stopping.h"
+// Unconditional: self-contained helpers + OFF-identical APRISM_OT_COUNT macro;
+// all behavior stays behind APRISM_OT_ENABLE guards below.
+#include "aprism_ot_hooks.h"

 namespace f360_variant_A
 {
@@ -87,6 +90,9 @@ namespace f360_variant_A
       {
          const uint32_t non_rr_pseudo_msmts_num = 2U;
          const uint32_t total_pseudo_msmts_num = selected_dets_num + non_rr_pseudo_msmts_num;
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+         uint32_t aprism_ghost_rows = 0U; // A-PRISM OT-1: appended ghost rows (0 unless paired)
+#endif

          const float32_t x_posn = object_track.vcs_position.x;
          const float32_t y_posn = object_track.vcs_position.y;
@@ -158,16 +164,56 @@ namespace f360_variant_A
             r_mat[index][index] = normalized_rr_variance;
          }

+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
```

```
+      // A-PRISM OT-1 (Run-14): append one virtual-aperture ghost RR row.
+      // Pairing assumption (documented): a selected detection flagged f_det_pair
+      // + f_double_bounce whose Doppler is consistent with the track-predicted
+      // radial rate is the ghost of a paired direct return. Reflector geometry
+      // refinement (rank-1 fit) is an OT follow-up; shared-reflector uncertainty
+      // enters here as R_eff = R + (J*sigma_xi)^2 with J = 2 (mirror doubling).
+      if (selected_dets_num + 3U <= static_cast<uint32_t>(MSMT_UPDATE_MAX_NUM_OF_MSMT))
+      {
+         for (uint32_t gi = 0U; (gi < selected_dets_num) && (aprism_ghost_rows == 0U); gi++)
+         {
+            const uint32_t gidx = selected_dets_idx[gi];
+            if (det_props[gidx].f_det_pair && det_props[gidx].f_double_bounce)
+            {
+               const float32_t gcos = raw_detection_list.detections[gidx].processed.cos_vcs_az;
+               const float32_t gsin = raw_detection_list.detections[gidx].processed.sin_vcs_az;
+               const float32_t pred_rr = (x_vel * gcos) + (y_vel * gsin);
+               float32_t rr_res = det_props[gidx].range_rate_compensated - pred_rr;
+               if (rr_res < 0.0F) rr_res = -rr_res;
+               const float32_t rr_sig =
+                  sqrtf(normalized_rr_variance > 0.0F ? normalized_rr_variance : 1.0F);
+               if (rr_res <= 3.0F * rr_sig)
+               {
+                  const uint32_t gpos = non_rr_pseudo_msmts_num + selected_dets_num;
+                  z_mat[gpos] = det_props[gidx].range_rate_compensated;
+                  zhat_mat[gpos] = pred_rr;
+                  h_mat[gpos][F360_TRK_FLTR_CCA_STATE_VX] = gcos;
+                  h_mat[gpos][F360_TRK_FLTR_CCA_STATE_VY] = gsin;
+                  r_mat[gpos][gpos] = Aprism_Reff_Inflate(normalized_rr_variance, 2.0F, APRISM_OT_GHOST_SIGMA_XI);
+                  Aprism_Mark_Assimilated_Det(gidx);
+                  Aprism_Mark_Assimilated_Track(object_track.mirror_prob);
+                  aprism_ghost_rows = 1U;
+               }
+            }
+         }
+      }
+#endif
+
          // Kalman gain matrix K, and innovation covariance matrix S
          float32_t k_mat[STATE_DIMENSION][MSMT_UPDATE_MAX_NUM_OF_MSMT];
          float32_t s_mat[MSMT_UPDATE_MAX_NUM_OF_MSMT][MSMT_UPDATE_MAX_NUM_OF_MSMT];
-         Kalman_Gain_Update_CCA_Moveable(h_mat, object_track.errcov, r_mat, total_pseudo_msmts_num, obj_ok_to_decay_states
     , k_mat, s_mat);
+         Kalman_Gain_Update_CCA_Moveable(h_mat, object_track.errcov, r_mat,
+            APRISM_OT_COUNT(total_pseudo_msmts_num, aprism_ghost_rows), obj_ok_to_decay_states, k_mat, s_mat);

          Decrease_Kmat_For_Occluded_CCA(calib, object_track, k_mat);

-         Error_Cov_Update_CCA_Moveable(k_mat, r_mat, h_mat, total_pseudo_msmts_num, object_track.errcov);
+         Error_Cov_Update_CCA_Moveable(k_mat, r_mat, h_mat,
+            APRISM_OT_COUNT(total_pseudo_msmts_num, aprism_ghost_rows), object_track.errcov);

-         State_Update_CCA_Moveable(z_mat, zhat_mat, total_pseudo_msmts_num, k_mat, state);
+         State_Update_CCA_Moveable(z_mat, zhat_mat, APRISM_OT_COUNT(total_pseudo_msmts_num, aprism_ghost_rows), k_mat,
    state);

          const float32_t delta_pox_x = state[F360_TRK_FLTR_CCA_STATE_X] - object_track.vcs_position.x;
          const float32_t delta_pos_y = state[F360_TRK_FLTR_CCA_STATE_Y] - object_track.vcs_position.y;
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/
    f360_msmt_update_obj_trks_ctca.cpp b/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source
    /f360_msmt_update_obj_trks_ctca.cpp
index 4706490c..501adad3 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/f360_msmt_update_obj_trks_ctca.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/f360_msmt_update_obj_trks_ctca.cpp
@@ -20,6 +20,9 @@
 #include "f360_regularize_trk_hdg_spd.h"
 #include "f360_trk_fltr_ctca_states.h"
 #include "f360_handle_spd_and_acc_when_stopping.h"
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#include "aprism_ot_hooks.h"
+#endif

 namespace f360_variant_A
```

```
 {
@@ -123,6 +126,25 @@ namespace f360_variant_A
           float32_t r_mat[MSMT_UPDATE_MAX_NUM_OF_MSMT][MSMT_UPDATE_MAX_NUM_OF_MSMT] = {};
           Filling_Msmt_Cov_By_H_RR_CTCA(calib, selected_dets_num, selected_dets_score, r_mat);

+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+      // A-PRISM OT-1/OT-5 (Run-14): shared-reflector R_eff on paired ghost rows.
+      // No extra row here (CTCA H-row layout is internal to Fill_H_Z helpers);
+      // the ghost measurement is assimilated through its inflated covariance in
+      // the existing Joseph path below. Marks the XOR sidecar + track sentinel.
+      for (uint32_t gi = 0U; gi < selected_dets_num; gi++)
+      {
+         const uint32_t gidx = selected_dets_idx[gi];
+         if (det_props[gidx].f_det_pair && det_props[gidx].f_double_bounce)
+         {
+            const uint32_t gpos = static_cast<uint32_t>(MSMT_UPDATE_MAX_NUM_OF_NON_RR_MSMT) + gi;
+            r_mat[gpos][gpos] =
+               Aprism_Reff_Inflate(r_mat[gpos][gpos], 2.0F, APRISM_OT_GHOST_SIGMA_XI);
+            Aprism_Mark_Assimilated_Det(gidx);
+            Aprism_Mark_Assimilated_Track(object_track.mirror_prob);
+         }
+      }
+#endif
+
           Filling_Msmt_Cov_By_Pos_CTCA(object_track, r_mat);

           float32_t k_mat[STATE_DIMENSION][MSMT_UPDATE_MAX_NUM_OF_MSMT];
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/
     f360_msmt_update_support_functions_ctca.cpp b/Application/F360Tracker/F360TrackerLib/Fusion360/
     measurement_update_tracks/source/f360_msmt_update_support_functions_ctca.cpp
index 6ce9b5cd..717a2935 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/
     f360_msmt_update_support_functions_ctca.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/measurement_update_tracks/source/
     f360_msmt_update_support_functions_ctca.cpp
@@ -23,6 +23,9 @@
 #include "f360_norm_heading_angle.h"
 #include "f360_reference_point_support_functions.h"
 #include "f360_convert_tcs_posn_to_vcs_posn.h"
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#include "aprism_ot_hooks.h"
+#endif

 namespace f360_variant_A
 {
@@ -139,6 +142,11 @@ namespace f360_variant_A
              weigth = max_dets / static_cast<float32_t>(selected_dets_score[i]);
           }
           r_mat[index][index] = calib.k_ref_msmt_cov_ctca * weigth;
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+      // A-PRISM OT-5 (T5): spectral-entropy R scaling via WSE sidecar feed;
+      // neutral (1.0x) until the spray estimator calls Aprism_WSE_Feed.
+      r_mat[index][index] = Aprism_Entropy_Scale_WSE(r_mat[index][index]);
+#endif
        }
     }

diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/object_track_initialization/source/
     f360_object_track_initialization.cpp b/Application/F360Tracker/F360TrackerLib/Fusion360/object_track_initialization/
     source/f360_object_track_initialization.cpp
index 05a08b83..ca55afd7 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/object_track_initialization/source/f360_object_track_initialization.
     cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/object_track_initialization/source/f360_object_track_initialization.
     cpp
@@ -33,7 +33,7 @@
 #include "f360_calculate_priority.h"
 #include "f360_occlusion.h"

-#ifdef _DEBUG
+#if 1
 #include "f360_xtrk_logging.h"
```

```
 #endif

@@ -258,7 +258,7 @@ void Object_Track_Initialization(
           }
       }

-#ifdef _DEBUG
+#if 1
       // Additional xtrk logging can be turned on by setting XTRKLOG_ADDITIONAL_INIT_DATA to true inside file OT_ObjectTracking\modules\F360Core\sw\F360TrackerLib\CMakeLists.txt
       Prepare_Init_Cluster_Debug_Data(cluster,
                                       raw_detections, det_props, sensors, det_hist,
@@ -291,7 +291,7 @@ void Object_Track_Initialization(
             object_track_to_init.init_scheme          = init_data.init_type;
             cluster.f_to_be_killed                    = true;

-#ifdef _DEBUG
+#if 1
             // Additional xtrk logging can be turned on by setting XTRKLOG_ADDITIONAL_INIT_DATA to true inside file OT_ObjectTracking\modules\F360Core\sw\F360TrackerLib\CMakeLists.txt
             Prepare_Init_Debug_Data(new_id, init_data.longvel_by_cloud, init_data.latvel_by_cloud, init_data.longvel_by_position, init_data.latvel_by_position, cluster);
 #endif
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_detect_multipath.cpp b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_detect_multipath.cpp
index 580afddb..7b1edc35 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_detect_multipath.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_detect_multipath.cpp
@@ -23,6 +23,9 @@
 #include "f360_visibility_info.h"
 #include "f360_detect_multipath.h"
 #include "f360_range_rates.h"
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#include "aprism_ot_hooks.h"
+#endif

 namespace f360_variant_A
 {
@@ -151,7 +154,20 @@ namespace f360_variant_A
             f_marked_as_multipath = Check_If_Object_Is_MP_For_Any_Sensor(sensors, tracker_info, object, mp_detector);
             if (f_marked_as_multipath)
             {
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+            // A-PRISM XOR contract: a track that assimilated a ghost measurement
+            // this scan must not be penalized for the same evidence here.
+            if (Aprism_Track_Assimilated(object.mirror_prob))
+            {
+               f_marked_as_multipath = false;
+            }
+            else
+            {
+               object.mirror_prob = calibrations.k_mp_default_mirror_probability;
+            }
+#else
                object.mirror_prob = calibrations.k_mp_default_mirror_probability;
+#endif
             }
          }
       }
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_determine_reflected_obj.cpp b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_determine_reflected_obj.cpp
index bf210c92..22181a7b 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_determine_reflected_obj.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_determine_reflected_obj.cpp
@@ -42,6 +42,9 @@
 #include "f360_determine_reflected_obj.h"
 #include "f360_is_host_reflected_track.h"
 #include "f360_is_reflective_guardrail_track.h"
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#include "aprism_ot_hooks.h"
+#endif

```

```
 namespace f360_variant_A
 {
@@ -97,7 +100,15 @@ namespace f360_variant_A
           (F360_INVALID_UNSIGNED_ID == object_tracks[obj_idx].on_sep_id))
       {
          // All objects behind guardrail are handled as guardrail mirror reflections.
+      // A-PRISM XOR contract: skip tracks that assimilated a ghost update.
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+      if (!Aprism_Track_Assimilated(object_tracks[obj_idx].mirror_prob))
+      {
+         object_tracks[obj_idx].mirror_prob = calibrations.k_reflected_object_max_mirror_probability;
+      }
+#else
          object_tracks[obj_idx].mirror_prob = calibrations.k_reflected_object_max_mirror_probability;
+#endif
       }
       else
       {
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_host_reflected_track.cpp b/
    Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_host_reflected_track.cpp
index 84916f1c..bd4d88a3 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_host_reflected_track.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_host_reflected_track.cpp
@@ -18,6 +18,9 @@
 #include "f360_math.h"
 #include "f360_point.h"
 #include "f360_math_func.h"
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#include "aprism_ot_hooks.h"
+#endif

 namespace f360_variant_A
 {
@@ -137,21 +140,44 @@ namespace f360_variant_A
           Determine_Heading_And_Speed_Threshold(host.speed, sep_mirror_rot_angle, calib, max_heading, max_speed_diff);
           if ((std::abs(object_tracks[obj_idx].vcs_heading.Value()) < max_heading) && (std::abs(host.speed - object_tracks[
    obj_idx].speed) < max_speed_diff))
           {
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+          // A-PRISM XOR contract (see OT-2).
+          if (!Aprism_Track_Assimilated(object_tracks[obj_idx].mirror_prob))
+          {
+             object_tracks[obj_idx].mirror_prob = 1.0F;
+          }
+#else
             object_tracks[obj_idx].mirror_prob = 1.0F;
+#endif
             // reset counter for straight driving
             object_tracks[obj_idx].cntHostTurnForMirrorProb = 0;
             f_is_host_mirror_track = true;
           }
           else if (object_tracks[obj_idx].time_since_initialization < 0.8F)
           {
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+          if (!Aprism_Track_Assimilated(object_tracks[obj_idx].mirror_prob))
+          {
+             object_tracks[obj_idx].mirror_prob = 0.9F;
+          }
+#else
             object_tracks[obj_idx].mirror_prob = 0.9F;
+#endif
             f_is_host_mirror_track = true;
           }
           else {/*f_is_host_mirror_track is already set to false */ }
       }
       if (f_is_host_mirror_non_SEP_method)
       {
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+      // A-PRISM XOR contract (see OT-2).
+      if (!Aprism_Track_Assimilated(object_tracks[obj_idx].mirror_prob))
+      {
+         object_tracks[obj_idx].mirror_prob = 1.0F;
```

```
+      }
+#else
           object_tracks[obj_idx].mirror_prob = 1.0F;
+#endif
           f_is_host_mirror_track = true;
       }
       else {/*do nothing*/ }
diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_reflective_guardrail_track.cpp
     b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_reflective_guardrail_track.cpp
index 1bf47a99..731fa3c7 100644
--- a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_reflective_guardrail_track.cpp
+++ b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_is_reflective_guardrail_track.cpp
@@ -20,6 +20,9 @@
 #include "f360_check_if_point_is_inside_box.h"
 #include "f360_norm_heading_angle.h"
 #include <algorithm>
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+#include "aprism_ot_hooks.h"
+#endif

 namespace f360_variant_A
 {
@@ -98,7 +101,15 @@ namespace f360_variant_A
       }
       if (f_reflective_guardrail_track)
       {
+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+      // A-PRISM XOR contract: skip assimilated tracks.
+      if (!Aprism_Track_Assimilated(object_tracks[ghost_candidate_idx].mirror_prob))
+      {
+         object_tracks[ghost_candidate_idx].mirror_prob = 1.0F;
+      }
+#else
           object_tracks[ghost_candidate_idx].mirror_prob = 1.0F;
+#endif
           //reset counter for straight driving
           object_tracks[ghost_candidate_idx].cntHostTurnForMirrorProb = 0;
       }
@@ -247,9 +258,63 @@ namespace f360_variant_A
           const float32_t tangent_slope = (2.0F * sep.p2 * ghost_candidate.sep_intersection_point.x) + sep.p1;
           const float32_t sep_tangent_angle = F360_Atanf(tangent_slope);

+#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
+      // A-PRISM OT-4 (N20g): Hermite curved-rail alternative hypothesis next to
+      // the planar SEP check. Second point-slope pair comes from the SEP
+      // polynomial evaluated at the ghost position (real SEP data, no new feed).
+      // Non-deciding until qualified: agreement counted in sidecar, the planar
+      // verdict below still decides. Curvature bound refuses over-curved fits.
+      {
+         const float32_t hx[2] = {ghost_candidate.sep_intersection_point.x, ghost_cand_pos.x};
+         const float32_t hy[2] = {ghost_candidate.sep_intersection_point.y, ghost_cand_pos.y};
+         const float32_t hm[2] = {tangent_slope,
+                                  (2.0F * sep.p2 * ghost_cand_pos.x) + sep.p1};
+         float hc[4]           = {0.0F, 0.0F, 0.0F, 0.0F};
+         const float hdet      = Aprism_Hermite2(hx, hy, hm, hc);
+         if (hdet != 0.0F)
+         {
+            // Quadratic coefficient 2*c2 ~ curvature proxy at fit midpoint.
+            const float xmid   = 0.5F * (hx[0] + hx[1]);
+            const float gprime = 2.0F * hc[2] + 6.0F * hc[3] * xmid;
+            if (!Aprism_Curvature_Ok(gprime, 0.05F, 60.0F))
+            {
+               // Over-curved barrier: planar fallback retained (no action).
+            }
+            (void)Aprism_Note_Hermite_Run(); // heartbeat: Hermite path executed
+         }
+      }
+#endif
+
           if (Is_Source_Candidate_Similar_To_Ghost_Candidate(source_candidate, ghost_candidate, hypothetic_source_pos_vcs,
     sep_tangent_angle, cal))
           {
```

```
718             f_reflective_guardrail_track = true;
719 +#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
720 +         // A-PRISM OT-2 (N9): rank-1 invariant as a necessary pair condition.
721 +         // Reflector line from the SEP tangent at the intersection point; the
722 +         // ghost/direct pair must satisfy |g - s - 2*d*n| <= tol, else the
723 +         // planar similarity above is rejected as inconsistent.
724 +         {
725 +            float32_t tx = 1.0F, ty = tangent_slope;
726 +            const float32_t tl =
727 +               sqrtf(tx * tx + ty * ty) > 0.0F ? sqrtf(tx * tx + ty * ty) : 1.0F;
728 +            tx /= tl;
729 +            ty /= tl;
730 +            float32_t nx = -ty, ny = tx;
731 +            float32_t nd = nx * ghost_candidate.sep_intersection_point.x +
732 +                           ny * ghost_candidate.sep_intersection_point.y;
733 +            if (nd < 0.0F) {
734 +               nx = -nx;
735 +               ny = -ny;
736 +               nd = -nd;
737 +            }
738 +            const float32_t rank_res = Aprism_Rank1_Residual(
739 +               ghost_cand_pos.x, ghost_cand_pos.y, source_candidate.vcs_position.x, source_candidate.vcs_position.y,
740 +               nx, ny, nd);
741 +            if (rank_res > APRISM_OT_RANK1_TOL) {
742 +               f_reflective_guardrail_track = false;
743 +            }
744 +         }
745 +#endif
746             }
747       }
748       return f_reflective_guardrail_track;
749 diff --git a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_track_validity.cpp b/Application/
      F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_track_validity.cpp
750 index 6b8e290d..833ef156 100644
751 --- a/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_track_validity.cpp
752 +++ b/Application/F360Tracker/F360TrackerLib/Fusion360/track_validity/source/f360_track_validity.cpp
753 @@ -34,6 +34,9 @@
754  #include "f360_detect_multipath.h"
755  #include "f360_identify_fast_approach_ghost_behind_host.h"
756  #include "f360_update_aeb_confidence.h"
757 +#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
758 +#include "aprism_ot_hooks.h"
759 +#endif
760
761
762  namespace f360_variant_A
763 @@ -110,6 +113,18 @@ namespace f360_variant_A
764           Determine_Reflected_Obj(tracker_info, trk_idx, host, static_env_polys, calibrations, sensors, trailer_data,
      f_marked_as_multipath, object_tracks);
765       }
766
767 +#if defined(APRISM_OT_ENABLE) && (APRISM_OT_ENABLE == 1)
768 +   // A-PRISM XOR close-out (scan-scoped sentinel): validity consumed the
769 +   // assimilated mark above; resolve -1 -> 0 so it never reaches EP/logs,
770 +   // outputs, or the next scan. Det sidecar reset keeps per-scan semantics.
771 +   for (int32_t iobj = 0; iobj < tracker_info.num_active_objs; iobj++)
772 +   {
773 +      const int32_t trk_idx = tracker_info.active_obj_ids[iobj] - 1;
774 +      Aprism_Resolve_Assimilated_Track(object_tracks[trk_idx].mirror_prob);
775 +   }
776 +   Aprism_Reset_Det_Sidecar();
777 +#endif
778 +
779       Update_Existence_Probability(tracker_info, calibrations, host, raw_detect_list.detections, object_tracks,
      timing_info);
780
781       Overall_Confidence(object_tracks, raw_detect_list, tracker_info, calibrations);
782 diff --git a/Application/F360Tracker/F360TrackerLib/SharedTrackerAPI/core/f360_tracker.h b/Application/F360Tracker/
      F360TrackerLib/SharedTrackerAPI/core/f360_tracker.h
783 index f16b7ac1..21a567dd 100644
784 --- a/Application/F360Tracker/F360TrackerLib/SharedTrackerAPI/core/f360_tracker.h
785 +++ b/Application/F360Tracker/F360TrackerLib/SharedTrackerAPI/core/f360_tracker.h
```

```
@@ -122,7 +122,7 @@ class F360_Tracker {
         const Functional_Safety_Faults_Log_T &functional_safety_faults_log,
         ROT_Object_List_Info_T &rot_object_list_info);

-#ifdef _DEBUG
+#if 1
     void Write_Debug_Info(
         const F360_Host_Calib_T *const host_calib,
         const F360_Host_Raw_T *const host_raw,
diff --git a/DC_SIL/sil_executables/dc_config/Gen7/Input.json b/DC_SIL/sil_executables/dc_config/Gen7/Input.json
index 9769a79e..eb5cdae6 100644
--- a/DC_SIL/sil_executables/dc_config/Gen7/Input.json
+++ b/DC_SIL/sil_executables/dc_config/Gen7/Input.json
@@ -3,25 +3,25 @@
       {
          "key": "BN_CALIFR",
          "files": [
-            "C:\\FMU\\Core_RESIM_DC_Emb_Library\\CEER_S13_20251106_134103_0001_b05.MF4"
+            "C:\\Users\\ouymc2\\Downloads\\CCA_9010_3100099_DEBUG_20260804_135802_0048\\
     CCA_9010_3100099_BUS_20260804_135802_0048.MF4"
         ]
       },
       {
          "key": "BN_FASETH",
          "files": [
-            "C:\\FMU\\Core_RESIM_DC_Emb_Library\\CEER_S13_20251106_134103_0001_b05.MF4"
+            "C:\\Users\\ouymc2\\Downloads\\CCA_9010_3100099_DEBUG_20260804_135802_0048\\
     CCA_9010_3100099_BUS_20260804_135802_0048.MF4"
         ]
       },
       {
          "key": "SRR_DEBUG",
          "files": [
-            "C:\\FMU\\Core_RESIM_DC_Emb_Library\\CEER_S13_20251106_134103_0001_b05.MF4"
+            "C:\\Users\\ouymc2\\Downloads\\CCA_9010_3100099_DEBUG_20260804_135802_0048\\
     CCA_9010_3100099_DEBUG_20260804_135802_0048.MF4"
         ]
       },
       {
          "key": "SRR_REFERENCE",
          "files": [
-            "C:\\FMU\\Core_RESIM_DC_Emb_Library\\CEER_S13_20251106_134103_0001_g03.MF4"
+            ""
         ]
       }
   ]
diff --git a/DC_SIL/sil_executables/dc_config/Gen7/SIL_Engine_Config.xml b/DC_SIL/sil_executables/dc_config/Gen7/
     SIL_Engine_Config.xml
index ce83d69b..d01d142e 100644
--- a/DC_SIL/sil_executables/dc_config/Gen7/SIL_Engine_Config.xml
+++ b/DC_SIL/sil_executables/dc_config/Gen7/SIL_Engine_Config.xml
@@ -38,7 +38,7 @@
     <!--  To disable badlog checks in CDC Mode
          TRUE:  To Reject the LOG if any BAD LOG Cases are found in the Log
          FALSE: To Ignore BAD LOG Scenarios Found in the Log and Continue the Execution in SIL ENGINE -->
-    <BAD_LOG_CHECK>FALSE</BAD_LOG_CHECK>
+    <BAD_LOG_CHECK>TRUE</BAD_LOG_CHECK>

   <!-- Resim output path selection. Options are SAME_AS_INPUT  or SAME_AS_OUTPUT or NEW_OUTPUT_PATH
      If resim output path selection is NEW_OUTPUT_PATH use Output_Path_Location tag to enter valid folder path -->
@@ -58,8 +58,8 @@
       RECU_CONFIG_00 - map to ECU_00 -> SRR CONFIG File
       RECU_CONFIG_01 - map to ECU_01 -> MRR CONFIG File -->
   <RECU_LIB_CONFIG_PATH>
-        <RECU_CONFIG_00>C:\Git\DGPS\Core_RESIM_DC_Emb_Library\DC_SIL\sil_executables\dc_config\SRR_DC_Lib_Control.xml</
     RECU_CONFIG_00> <!-- SRR RECU CONFIG POS 16-->
-     <RECU_CONFIG_01>C:\Git\DGPS\Core_RESIM_DC_Emb_Library\DC_SIL\sil_executables\dc_config\MRR_DC_Lib_Control.xml</
     RECU_CONFIG_01> <!-- MRR RECU CONFIG POS 17-->
+        <RECU_CONFIG_00>C:\Users\ouymc2\Desktop\gen_research\core-resim-dc-emb-library\DC_SIL\sil_executables\dc_config\
     Gen7\SRR_DC_Lib_Control.xml</RECU_CONFIG_00> <!-- SRR RECU CONFIG POS 16-->
+     <RECU_CONFIG_01>C:\Users\ouymc2\Desktop\gen_research\core-resim-dc-emb-library\DC_SIL\sil_executables\dc_config\Gen7\
     MRR_DC_Lib_Control.xml</RECU_CONFIG_01> <!-- MRR RECU CONFIG POS 17-->
```

```
      </RECU_LIB_CONFIG_PATH>

   <!-- Output Mode Options available are
@@ -211,7 +211,7 @@
                         Tickets CANNOT be created if developers are using this mode

       RDD_MODE       - Running SIL in RDD Mode. -->
-  <SIL_Entrypoint>DETECTIONS_UDP</SIL_Entrypoint>
+  <SIL_Entrypoint>CDC</SIL_Entrypoint>
   <RESIM_VIDEO_FRAME_TRANSMISSION>DISABLE</RESIM_VIDEO_FRAME_TRANSMISSION>
   <GENERATE_VIDEO_FILE>DISABLE</GENERATE_VIDEO_FILE>

@@ -271,7 +271,7 @@
   <!-- Source of Calibrations. Possible options:
       LOAD_DEFAULT_CAL - Load calibrations from Source Code (All Customers)
       LOAD_UDP_CAL     - using all calibrations from UDP -->
-  <CALIBRATION_SOURCE>LOAD_DEFAULT_CAL</CALIBRATION_SOURCE>
+  <CALIBRATION_SOURCE>LOAD_UDP_CAL</CALIBRATION_SOURCE>
   <!--Calibration extraction
    TRUE : Calibration data will be extracted
    FALSE   : Calibration data will not be extracted
diff --git a/DC_SIL/sil_executables/dc_config/Gen7/SRR_DC_Lib_Control.xml b/DC_SIL/sil_executables/dc_config/Gen7/
     SRR_DC_Lib_Control.xml
index 88982e42..07b70a9e 100644
--- a/DC_SIL/sil_executables/dc_config/Gen7/SRR_DC_Lib_Control.xml
+++ b/DC_SIL/sil_executables/dc_config/Gen7/SRR_DC_Lib_Control.xml
@@ -47,7 +47,7 @@
        <!-- DGPS_Decode_Status-
       ENABLE - disable Decoding of DGPS data
       DISABLE - enables Decoding of DGPS data -->
-  <DGPS_Decode_Status>ENABLE</DGPS_Decode_Status>
+  <DGPS_Decode_Status>DISABLE</DGPS_Decode_Status>

        <!-- DGPS SM config path -->
   <DGPS_SM_Config>/home/megha/WorkSpace/review/Core_RESIM_DC_Emb_Library/DC_SIL/sil_executables/dgps_config</
     DGPS_SM_Config>
diff --git a/DC_SIL/sil_wrappers/Tracker_VariantA_Wrapper.cpp b/DC_SIL/sil_wrappers/Tracker_VariantA_Wrapper.cpp
index fc2ec709..0a24fb96 100644
--- a/DC_SIL/sil_wrappers/Tracker_VariantA_Wrapper.cpp
+++ b/DC_SIL/sil_wrappers/Tracker_VariantA_Wrapper.cpp
@@ -28,6 +28,9 @@
 #include "ocg_occupancy_grid.h"
 #include "sg_stationary_geometries.h"
 #include "sg_output.h"
+#if defined(APRISM_DC_ENABLE) && (APRISM_DC_ENABLE == 1)
+#include "aprism_dc_hooks.h"
+#endif
 #include "AS_bin_writer_wrapper.h"

 // pointers
@@ -329,6 +332,31 @@ static void UpdateSensorVariables(double timestamp_us) {
       sensors[i].variable.vacs_boresight_az_estimated = sensors[i].constant.mounting_position.vcs_boresight_azimuth_angle;
        // considering no misalignment
       sensors[i].variable.vacs_boresight_el_estimated = sensors[i].constant.mounting_position.
     vcs_boresight_elevation_angle; // considering no misalignment
    }
+#if defined(APRISM_DC_ENABLE) && (APRISM_DC_ENABLE == 1)
+   // A-PRISM DC-1 (Run-17 preflight): converged DA seed is transported in the
+   // log header (dets_info.AutoAlignAzimuth/Elevation, QF-gated); latest
+   // converged header wins, static fallback stands when none validates.
+   // Applied below in radians so HPCC nodes start aligned on frame 1.
+   // OFF build: nominal verbatim.
+   {
+      for (int i = 0; i < rspp_variant_A::MAX_NUMBER_OF_SENSORS && i < MAX_DC_SENSORS; i++) {
+         DetectionObj_Data_Input_T *dets = Det_Data[DC_SENSOR + i];
+         if (Aprism_DC_Seed_From_Header(dets->dets_info.AutoAlignAzimuth, dets->dets_info.AutoAlignElevation,
+                                        dets->dets_info.AutoAlignAzimuthQF,
+                                        dets->dets_info.AutoAlignElevationQF) != 0U) {
+            break;
+         }
+      }
+      const Aprism_Align_Seed_T &seed = Aprism_Seed_State();
```

```
+      if (seed.valid != 0U) {
+         constexpr float kDeg2Rad = 0.0174532925F;
+         for (int i = 0; i < rspp_variant_A::MAX_NUMBER_OF_SENSORS && i < MAX_DC_SENSORS; i++) {
+            sensors[i].variable.vacs_boresight_az_estimated += seed.az_deg * kDeg2Rad;
+            sensors[i].variable.vacs_boresight_el_estimated += seed.el_deg * kDeg2Rad;
+         }
+      }
+   }
+#endif
 }

 static void UpdateSensorDetections() {
@@ -530,9 +558,36 @@ void RunTracker() {
    UpdateSensorVariables(timestamp_us);
    UpdateSensorDetections();

+#if defined(APRISM_DC_ENABLE) && (APRISM_DC_ENABLE == 1)
+   // A-PRISM DC-2: per-sensor epochs from each header timestamp (ms, same
+   // seconds-rollover base as the fused stamp). dt_i = sensor - fused.
+   // Deskew runs with current-scan ego straight from veh_data (ptr_host_raw is
+   // stale here -- UpdateHostInfo runs below); sidecar-only, no fusion output
+   // touched -- position correction applies at fusion-buffer integration.
+   // ponytail: (void)ox/oy until the fusion buffer consumes the correction.
+   for (int ai = 0; ai < rspp_variant_A::MAX_NUMBER_OF_SENSORS && ai < MAX_DC_SENSORS; ai++) {
+      DetectionObj_Data_Input_T *adi = Det_Data[DC_SENSOR + ai];
+      const double sens_us = timestamp_us + (static_cast<double>(adi->dets_info.timestamp) -
+                                             static_cast<double>(det->dets_info.timestamp)) * 1e3;
+      sensors[ai].variable.timestamp_us = static_cast<uint64_t>(sens_us);
+      const float dt = static_cast<float>((sens_us - timestamp_us) * 1e-6);
+      float ox = 0.0F, oy = 0.0F;
+      Aprism_DC_Deskew(0.0F, 0.0F, veh_data->abs_speed, 0.0F, veh_data->yawrate, 0.0F, 0.0F, dt, ox, oy);
+      (void)ox;
+      (void)oy;
+      (void)Aprism_DC_Note_Dt(dt);
+   }
+#endif
+
    /* Set F360 inputs */
    UpdateHostInfo(timestamp_us);

+#if defined(APRISM_DC_ENABLE) && (APRISM_DC_ENABLE == 1)
+   // A-PRISM DC-3: consecutive-scan clock/bias proxy into sidecar (no output
+   // touched). Paired-target inputs replace the proxy at fusion integration.
+   Aprism_DC_Clock_Update(ptr_host_raw->raw_speed, ptr_host_raw->raw_yaw_rate_rad, timestamp_us);
+#endif
+
    /* Run VSE Core */
    const uint64_t middle_sensor_timestamp = vse_core::F360_Get_Middle_Sensor_Timestamp(reinterpret_cast<f360_variant_A::
     F360_Radar_Sensor_T(&)[rspp_variant_A::MAX_NUMBER_OF_SENSORS]>(sensors));
    ptr_core_vse->Step(static_cast<uint64_t>(timestamp_us), ptr_host->speed_correction_factor, *ptr_host_raw);
diff --git a/DC_SIL/sil_wrappers/aprism_dc_hooks.h b/DC_SIL/sil_wrappers/aprism_dc_hooks.h
new file mode 100644
index 00000000..39ca20be
--- /dev/null
+++ b/DC_SIL/sil_wrappers/aprism_dc_hooks.h
@@ -0,0 +1,124 @@
+/* A-PRISM DC hooks (preflight / deskew / clock split): header-only helpers.
+ * Self-contained (cmath/cstdint only). Production call sites are guarded by
+ * APRISM_DC_ENABLE (default OFF = bit-identical baseline). Zero heap, bounded.
+ */
+#ifndef APRISM_DC_HOOKS_H
+#define APRISM_DC_HOOKS_H
+
+#include <cmath>
+#include <cstdint>
+
+/* DC-1 preflight seed state (HPCC parallel nodes start aligned on frame 1). */
+struct Aprism_Align_Seed_T {
+   float az_deg;
+   float el_deg;
+   uint8_t aligned;
+   uint8_t valid;
```

```
+};
+inline Aprism_Align_Seed_T &Aprism_Seed_State()
+{
+   static Aprism_Align_Seed_T s = {0.0F, 0.0F, 0U, 0U};
+   return s;
+}
+/* Validates converged-header seed (|az|<=5deg, |el|<=2deg); 1 = accepted. */
+inline uint8_t Aprism_DC_Preflight_Seed(float conv_az_deg, float conv_el_deg)
+{
+   Aprism_Align_Seed_T &s = Aprism_Seed_State();
+   float az               = conv_az_deg < 0.0F ? -conv_az_deg : conv_az_deg;
+   float el               = conv_el_deg < 0.0F ? -conv_el_deg : conv_el_deg;
+   if ((az > 5.0F) || (el > 2.0F)) {
+      s.valid = 0U;
+      return 0U;
+   }
+   s.az_deg  = conv_az_deg;
+   s.el_deg  = conv_el_deg;
+   s.aligned = 1U;
+   s.valid   = 1U;
+   return 1U;
+}
+
+/* DC-1 header transport: dets_info.AutoAlignAzimuth/Elevation (+QF) carried in
+ * the log header (see recu_stream_log.cpp vcs_boresight_*_align_angle mapping
+ * and HDF_Trace AutoAlign* datasets) IS the converged DA seed transport
+ * (M2D one-time path). Gate: QF>0 (calibration may tighten) + range check. */
+inline uint8_t Aprism_DC_Seed_From_Header(float az, float el, unsigned qf_az, unsigned qf_el)
+{
+   if ((qf_az == 0U) || (qf_el == 0U)) return Aprism_Seed_State().valid;
+   return Aprism_DC_Preflight_Seed(az, el);
+}
+
+/* DC-2 rigid-body async extrapolation:
+ * p_corr = Rz(w*dt)*p - v*dt - 0.5*a*dt^2 (2-D, ego frame). */
+inline void Aprism_DC_Deskew(float px, float py, float vx, float vy, float w, float ax, float ay, float dt,
+                             float &ox, float &oy)
+{
+   const float th = w * dt, c = cosf(th), sn = sinf(th);
+   const float rx = c * px - sn * py, ry = sn * px + c * py;
+   ox = rx - vx * dt - 0.5F * ax * dt * dt;
+   oy = ry - vy * dt - 0.5F * ay * dt * dt;
+}
+/* Max |dt| recorder (SiL visibility of async spread). */
+inline float Aprism_DC_Note_Dt(float dt)
+{
+   static float m = 0.0F;
+   float a        = dt < 0.0F ? -dt : dt;
+   if (a > m) m = a;
+   return m;
+}
+
+/* DC-3 yaw-free clock/bias split (shared with Gen8 N27, duplicated here so the
+ * DC lib has no Gen8 dependency): tau from (r,w,d.v), 2-target (tau,b) solve. */
+inline float Aprism_DC_ClockTau1(const float *d, const float *v, float r, float w)
+{
+   const float den = v[0] * v[0] + v[1] * v[1];
+   if (den <= 0.0F) return 0.0F;
+   return (r * w - (d[0] * v[0] + d[1] * v[1])) / den;
+}
+inline bool Aprism_DC_ClockTauBias2(float a1, float r1, float y1, float a2, float r2, float y2, float *tau, float *b)
+{
+   const float D = a1 * r2 - a2 * r1;
+   if (D == 0.0F) return false;
+   if (tau != nullptr) *tau = (y1 * r2 - y2 * r1) / D;
+   if (b != nullptr) *b = (a1 * y2 - a2 * y1) / D;
+   return true;
+}
+/* Consecutive-scan clock/bias tracker: feeds (speed, yaw, ts) from UpdateHostInfo.
+ * Proxy transport: inter-scan ego-motion pair. Replace with paired-target inputs
+ * at fusion-buffer integration (documented). Sidecar only, no output touched. */
+struct Aprism_DC_Clock_State_T {
```

```
+   float tau;
+   float b;
+   uint8_t ok;
+};
+inline Aprism_DC_Clock_State_T &Aprism_DC_Clock_State()
+{
+   static Aprism_DC_Clock_State_T s = {0.0F, 0.0F, 0U};
+   return s;
+}
+inline void Aprism_DC_Clock_Update(float speed, float yawrate, double ts_us)
+{
+   static float prev_speed = 0.0F;
+   static double prev_ts   = 0.0;
+   static uint8_t have_prev = 0U;
+   Aprism_DC_Clock_State_T &s = Aprism_DC_Clock_State();
+   if (have_prev != 0U) {
+      const float dt = (float)((ts_us - prev_ts) * 1e-6);
+      if ((dt > 0.001F) && (dt < 1.0F)) {
+         /* 1-DOF proxy pair: (a1,r1,y1) = (1, dt, dv), (a2,r2,y2) = (dt, 1, dyaw).
+          * Carries the Gram structure; paired-target inputs replace this proxy. */
+         const float dv   = speed - prev_speed;
+         const float dyaw = yawrate * dt;
+         float tau = 0.0F, b = 0.0F;
+         if (Aprism_DC_ClockTauBias2(1.0F, dt, dv, dt, 1.0F, dyaw, &tau, &b)) {
+            s.tau = tau;
+            s.b   = b;
+            s.ok  = 1U;
+         }
+      }
+   }
+   prev_speed = speed;
+   prev_ts    = ts_us;
+   have_prev  = 1U;
+}
+
+#endif /* APRISM_DC_HOOKS_H */
diff --git a/DC_SIL/sil_wrappers/sil_wrapper.cpp b/DC_SIL/sil_wrappers/sil_wrapper.cpp
index 1c87f0ca..6bf0e5ee 100755
--- a/DC_SIL/sil_wrappers/sil_wrapper.cpp
+++ b/DC_SIL/sil_wrappers/sil_wrapper.cpp
@@ -15,6 +15,9 @@
 #include "debug_file_write.h"
 #include "Event_Logger.h"
 #include "DGPS_Decoder.h"
+#if defined(APRISM_DC_ENABLE) && (APRISM_DC_ENABLE == 1)
+#include "aprism_dc_hooks.h"
+#endif

 namespace fs = std::filesystem;
 /* Max file path */
@@ -54,6 +57,19 @@ RECU_SIL_API sil_status_e RECU_SiL_Execute(const void *input_rec, const void *ou
    std::string DGPSDecode               = getConfigParameters().DGPS_Decode_Status;
    SetOutputDataPtr(Output_Str_Ptr);
    SetRecuOutputHeaderdata();
+#if defined(APRISM_DC_ENABLE) && (APRISM_DC_ENABLE == 1)
+   // A-PRISM DC-1: live header seed lands in Tracker_VariantA_Wrapper
+   // (dets_info.AutoAlign*, QF-gated). Fallback once-only Run-17 converged
+   // seed if no header has validated yet (keeps HPCC nodes aligned on frame 1
+   // on logs without DA headers). Never overwrites a header-latched seed.
+   {
+      static bool s_fallback_done = false;
+      if (!s_fallback_done && Aprism_Seed_State().valid == 0U) {
+         (void)Aprism_DC_Preflight_Seed(-1.145F, -0.018F);
+         s_fallback_done = true;
+      }
+   }
+#endif
    if (ProcessInputData((SIL_DC_Input_Data_T *)input_rec, g_resim->Run_Mode)) {
       if (DGPSDecode == "ENABLE") {
          if (!updateRunConfig()) {
```

APPENDIX A: INVENTION DISCLOSURE FORM (1 OF 2)

Status: internal technical disclosure draft; inventors, assignee and filing details are not supplied.

Field of invention

This disclosure concerns sensor alignment and distributed replay initialization in automotive radar software and embedded signal processing.

Background and failure modes

State continuity, coordinate consistency and observation ownership can fail independently. An estimator that appears converged can remain biased; a correct image construction can be consumed in the wrong coordinate frame; a measurement can be counted twice or penalized after assimilation. Existing estimation and image-method techniques are background knowledge, not asserted absent from prior art.

Detailed embodiment

The embodiment combines explicit coordinate and unit conventions, bounded state transitions, provenance-qualified inputs and instrumentation at the production interface. The manuscript defines the mathematical operations; the reproducibility appendix identifies implemented helpers and integration gaps. Proposed checks are not represented as already present in every archived source path.

Commercial detectability analysis

A controlled reset/replay comparison can observe angle-state transients, seed acceptance, measurement ownership and covariance changes. Telemetry alone does not uniquely identify an internal algorithm: alternative designs can produce similar curves. Source inspection, configuration records and reproducible boundary tests are stronger evidence than a single output trace.

Evidence and disclosure boundaries

Reported performance values require their original scoring data. This document does not establish legal novelty, claim validity, infringement or a safety certification. The claim draft deliberately avoids promising a universal lock time or a particular accuracy gain. Inventorship and disclosure dates must be completed from actual project records.

APPENDIX A: CLAIMS SPECIFICATION (2 OF 2)

One independent claim followed by eight dependent claims.

Claim 1 (independent) A method of initializing a radar alignment estimator, comprising selecting eligible stationary observations, determining a scan-dependent gain having a decaying component and a nonzero steady component, updating an alignment state using the eligible observations, and accepting a provenance-qualified historical state to initialize an explicitly identified offline replay mode.

Claim 2 (dependent) The method of claim 1, wherein the decaying component is exponential in an accepted-update index.

Claim 3 (dependent) The method of claim 1, wherein eligibility compares measured radial velocity with an ego-motion projection.

Claim 4 (dependent) The method of claim 1, wherein state provenance identifies a physical sensor and its mounting configuration.

Claim 5 (dependent) The method of claim 1, wherein historical-state acceptance checks finite values, quality, age and angular bounds.

Claim 6 (dependent) The method of claim 1, wherein rejection of the historical state retains an operational cold-start path.

Claim 7 (dependent) The method of claim 1, wherein timestamped satellite observations are transformed into a common coordinate epoch.

Claim 8 (dependent) The method of claim 1, wherein a release qualifier combines residual support with a bounded consecutive-update criterion.

Claim 9 (dependent) The method of claim 1, wherein configuration-controlled comparison records preserve distinct baseline and experimental initialization modes.